CONTAINER ve DOCKER ile UYGULAMA PAKETLEME ve ÇALIŞTIRMA

İÇİNDEKİLER

CONTAINER ve DOCKER ile UYGULAMA PAKETLEME ve ÇALIŞTIRMA 1
Tablolar Listesi 2
Şekiller Listesi 2
1. Dersin Amacı ve Öğrenme Çıktıları 3
1.1. Dersin amacı 3
1.2. Öğrenme Çıktıları 3
2. Container Teknolojisinin Temelleri 3
2.1. Paketleme Gereksinimi ve Container Çalışma Yaklaşımı 3
2.2. Sanal Makine Karşılaştırması, İzolasyon ve Taşınabilirlik 4
3. Çalışma Ortamları ve Docker Kurulumu 5
3.1. Docker Engine, Docker Desktop ve Docker Compose'un Görevleri 5
3.2. Yerel ve bulut çalışma seçenekleri 6
4. İmajlar ve Container Yaşam Döngüsü 6
4.1. İmaj, Container ve Katman İlişkisi 6
4.2. Registry, Repository, Tag ve Digest 8
5. Dockerfile ile Uygulama İmajı Oluşturma 12
5.1. Dockerfile'ın Görevi ve Temel Yönergeler 12
5.2. Derleme Bağlamı ve İmaj Oluşturma Süreci 13
5.3. İmajı Doğrulama ve Uygulamayı Güncelleme Yaklaşımı 15
6. Container Verisi ve Ağ Bağlantıları 16
6.1. Container Dosya Sistemi ve Kalıcı Veri Yönetimi 16
6.2. Container Ağı, Port Yayımlama ve Servisler Arası Bağlantı 18
6.3. Veri ve Bağlantı Sorunlarını Anlama 19
7. Docker Compose ile Çok Bileşenli Uygulama Yönetimi 21
7.1. Compose'un Görevi ve Uygulama Bileşenlerinin Tanımlanması 21
7.2. Birlikte Başlatma, Bağımlılıklar ve Hazır Olma 23
7.3. Uygulamayı İnceleme, Güncelleme ve Veriyi Koruyarak Sonlandırma 24
8. Container İmajlarının Bulutta Kullanılması 26
8.1. İmajı Registry Üzerinden Yerelden Buluta Taşıma 26
8.2. Bulut Sanal Makinesinde Çalıştırma ve Yönetilen Hizmet Seçenekleri 27

8.3. AWS, Microsoft Azure ve Google Cloud'da Kavramların Karşılıkları....29
8.4. Bulutta Erişim, Yapılandırma, Kalıcı Veri ve Kaynakların Sonlandırılması....29
9. Container Güvenliğinin Temelleri....31
9.1. İmaj Güvenliği ve Kullanıcı Yetkileri....31
9.2. Hassas Bilgiler ve Güvenlik Kontrolleri....32
10. Uygulama Çalışması....34
10.1. Uygulamanın Amacı ve Kapsamı....34
10.2. Yerel Uygulama....34
10.3. Bulut Uygulaması....35
10.4. Doğrulama ve Teslim....36
10.5. Sonraki Haftaya Hazırlık....37
11. Bölüm Özeti....38
Kaynakça....39
Hafta 5 – Çalışma Soruları....42

Tablolar Listesi

Tablo 1. Sanal makine ve olağan Linux container yaklaşımının karşılaştırılması. 4
Tablo 2. Docker araçlarının temel görevleri. 5
Tablo 3. Yerel ve bulut ortamında container çalıştırma seçenekleri. 6
Tablo 4. Örnek imaj adresinin bileşenleri. 9
Tablo 5. Temel Dockerfile yönergeleri ve görevleri. 12
Tablo 6. Örnek proje dosyalarının derleme bağlamı ve imaj içeriği açısından değerlendirilmesi. 14
Tablo 7. Container verisinin tutulduğu alanların karşılaştırılması. 17
Tablo 8. Tarayıcı erişimi ile servisler arası bağlantının karşılaştırılması. 19
Tablo 9. Veri ve bağlantı sorunlarında başlangıç kontrolleri. 20
Tablo 10. Temel Compose alanları ve uygulamadaki görevleri. 21
Tablo 11. Servislerin çalışma ve doğrulama düzeyleri. 24
Tablo 12. Compose uygulamasını sonlandırma seçeneklerinin kaynaklara etkisi. 25
Tablo 13. Sanal makinede çalışma ile yönetilen container hizmetlerinin karşılaştırılması....28
Tablo 14. Derste öğrenilen kavramların üç bulut sağlayıcısındaki hizmet karşılıkları. 29
Tablo 15. Uygulama çalışmasında doğrulanması gereken sonuçlar. 36

Şekiller Listesi

Şekil 1. Aynı imajın salt okunur katmanlarını kullanan container'ların ayrı yazılabilir katmanları. .8
Şekil 2. Container'ın temel yaşam döngüsü ve durum geçişleri. 11
Şekil 3. Compose ile tanımlanan web uygulaması, veri servisi, ortak ağ ve kalıcı veri alanı. 22

1. Dersin Amacı ve Öğrenme Çıktıları

1.1. Dersin amacı

Bu haftanın amacı, öğrencinin container (konteyner) yaklaşımını kavraması, hazır bir uygulamayı Docker ile imaja dönüştürmesi ve ağ bağlantıları ile veri kalıcılığını dikkate alarak çalıştırabilmesidir. Yerel uygulamalar üzerinden kazanılan becerilerin, aynı imajın bulut ortamında kullanılmasına temel oluşturması hedeflenmektedir.

1.2. Öğrenme Çıktıları

Bu bölümün sonunda öğrenci:

1. Container ile sanal makineyi karşılaştırabilir; imaj ve container arasındaki ilişkiyi açıklayabilir.
2. Basit bir Dockerfile kullanarak uygulama imajı oluşturabilir; container'ı çalıştırabilir, inceleyebilir ve sonlandırabilir.
3. Ağ bağlantılarını ve kalıcı veri alanlarını yapılandırabilir; Docker Compose ile uygulama bileşenlerini birlikte çalıştırabilir.
4. Temel güvenlik önlemlerini dikkate alarak bir imajın yerel ortamdan buluta taşınması ve çalıştırılması için gereken adımları açıklayabilir.

2. Container Teknolojisinin Temelleri

2.1. Paketleme Gereksinimi ve Container Çalışma Yaklaşımı

Bir uygulamanın çalışması yalnız kaynak koduna bağlı değildir. Uygun çalışma zamanı, kütüphaneler ve yapılandırma da gerekir. Örneğin bir Python uygulaması, geliştiricinin bilgisayarında çalışırken sunucuda farklı bir Python sürümü veya eksik bir kütüphane nedeniyle hata verebilir. Her ortamın elle hazırlanması, kurulum farklılıklarını ve bağımlılık çakışmalarını artırabilir.

Bir uygulamanın çalışması için kaynak kodunun yanında uygun çalışma zamanı, kütüphaneler ve yapılandırma gerekir. Örneğin bir Python uygulaması, geliştiricinin bilgisayarında çalışırken bulut sunucusunda eksik bir kütüphane veya farklı bir sürüm nedeniyle hata verebilir. Ortamların elle hazırlanması, bu farklılıkların tespit edilmesini ve giderilmesini zorlaştırır.

Container yaklaşımı, uygulamayı ihtiyaç duyduğu yazılım bileşenleriyle paketleyerek daha tutarlı bir çalışma ortamı sağlar. Uygulama kodu, çalışma zamanı ve gerekli kütüphaneler bir image (imaj) içinde bir araya getirilir. Bu imajdan oluşturulan container, uygulamayı izole bir süreç ortamında çalıştırır [1], [2].

Örneğin bir görev listesi uygulamasını paketlerken uygulama dosyaları, Python çalışma zamanı ve kullanılan kütüphaneler imaja eklenir. Uygulamayı başka bir makinede çalıştırmak için bu bileşenleri tek tek kurmak yerine, uyumlu bir container çalıştırma

ortamında aynı imaj kullanılır. Docker, imaj oluşturmak ve container'ları yönetmek için kullanılan araçlardan biridir.

Bu yaklaşımda imaj ile container farklı kavramlardır: İmaj uygulamanın paketini, container ise bu paketten oluşturulan çalışma örneğini temsil eder. Aynı imajdan birden fazla container oluşturulabilir [2].

Olağan Linux container'ları, üzerinde çalıştıkları Linux sisteminin kernel (çekirdek) bileşenini paylaşır. Her container için ayrı bir çekirdek başlatılmaz; uygulama süreçleri ve kaynak görünümleri işletim sistemi mekanizmalarıyla ayrılır [1].

Paketleme, ağ bağlantılarını ve veri kalıcılığını kendiliğinden sağlamaz. Örneğin görev listesi uygulamasının kullanıcılar tarafından erişilebilir olması ve kaydedilen görevlerin container değiştirildiğinde korunması ayrıca yapılandırılmalıdır. Bu nedenle uygulamayı çalıştırırken paketin içeriği, çalışma ayarları ve kalıcı veriler birlikte değerlendirilir.

2.2. Sanal Makine Karşılaştırması, İzolasyon ve Taşınabilirlik

Virtual Machine (sanal makine, VM), bir hypervisor (hipervizör) tarafından sağlanan sanal donanım üzerinde kendi işletim sistemini ve çekirdeğini çalıştırır. Olağan Linux container'ları ise üzerinde çalıştıkları Linux sisteminin çekirdeğini paylaşır. Bu nedenle container oluşturulurken ayrı bir işletim sistemi başlatılması gerekmez; uygulama ve gerekli yazılım bileşenleri izole bir ortamda çalıştırılır [1]. Temel farklar Tablo 1'de özetlenmiştir.

Tablo 1. Sanal makine ve olağan Linux container yaklaşımının karşılaştırılması.

Özellik	Sanal makine	Container
Temel yaklaşım	Donanımın sanallaştırılması	Uygulama süreçlerinin izole edilmesi
İşletim sistemi çekirdeği	Her VM kendi çekirdeğini çalıştırır	Aynı Linux sistemi üzerindeki container'lar çekirdeği paylaşır
Kaynak gereksinimi	Konuk işletim sistemi nedeniyle genellikle daha yüksek	Ayrı çekirdek başlatılmadığı için genellikle daha düşük
Başlatma	Konuk işletim sisteminin açılmasını içerir	Uygulama süreçlerinin başlatılmasına dayanır
Kullanım örneği	Ayrı işletim sistemi gerektiren ortam	Uygulamanın bağımlılıklarıyla paketlenmesi

Bu iki yaklaşım birlikte kullanılabilir. Örneğin bir bulut VM'si üzerinde Docker çalıştırılarak birden fazla uygulama container'ı yönetilebilir. VM altyapı ortamını, container'lar ise uygulamaların çalışma ortamlarını sağlar [1].

İzolasyon, uygulamaların süreç, dosya sistemi ve ağ görünümlerinin ayrılmasını sağlar. Linux'ta namespace (ad alanı) mekanizmaları bu ayrımı oluştururken control groups

(kontrol grupları, cgroups) kaynak kullanımının ölçülmesini ve sınırlandırılmasını sağlar. Örneğin bir container'ın bellek tüketimine sınır konulabilir [3].

Ancak container izolasyonu mutlak bir güvenlik garantisi değildir. Paylaşılan çekirdek, verilen yetkiler ve container'a bağlanan ana makine dizinleri güvenliği etkiler. Bu nedenle uygulamanın yalnız ihtiyaç duyduğu kaynaklara ve yetkilere erişmesi sağlanmalıdır [3].

Taşınabilirlik, uygulamanın bağımlılıklarını her ortamda yeniden kurmak yerine aynı imajı kullanabilmesidir. Bunun için hedef ortamın işletim sistemi ve işlemci mimarisiyle uyumluluk gerekir. Örneğin `linux/amd64` için hazırlanmış bir imaj, `linux/arm64` ortamında doğrudan çalışmayabilir; uygun imaj sürümü veya emülasyon gerekebilir [4].

Ayrıca yerelde kullanılan dosya yolları, ağ adresleri ve erişim bilgileri bulutta farklı olabilir. Dolayısıyla aynı imajın taşınması, bütün çalışma ayarlarının aynı kalacağı anlamına gelmez. İmaj uygulama paketini taşır; hedef ortama ait bağlantılar ve veri alanları ayrıca yapılandırılır.

3. Çalışma Ortamları ve Docker Kurulumu

3.1. Docker Engine, Docker Desktop ve Docker Compose'un Görevleri

Docker ile çalışırken kullanılan üç temel araç farklı görevler üstlenir. Docker Engine, imajları, container'ları, ağları ve veri alanlarını yöneten çalışma altyapısıdır. Kullanıcı, Command Line Interface (komut satırı arayüzü, CLI) üzerinden docker komutlarını verir. Bu komutlar, arka planda çalışan Docker servisine iletilir [5].

Docker Desktop, yerel geliştirme için Docker Engine, komut satırı araçları ve Docker Compose'u bir arada sunar. Grafik arayüzü sayesinde container durumları ve imajlar görüntülenebilir. Ancak bu derste, işlemlerin farklı ortamlarda tekrar edilebilmesi için ağırlıklı olarak komut satırı kullanılacaktır [6].

Docker Compose, birden fazla container'dan oluşan uygulamanın bileşenlerini birlikte tanımlayıp yönetir. Örneğin web uygulaması ile veri servisi; ağ ve kalıcı veri ayarlarıyla birlikte bir dosyada belirtilebilir [7]. Araçların görevleri Tablo 2'de özetlenmiştir.

Tablo 2. Docker araçlarının temel görevleri.

Araç	Temel görevi	Bu haftadaki kullanımı
Docker Engine	Container çalışma altyapısını sağlamak	İmaj ve container işlemlerini yürütmek
Docker Desktop	Yerel geliştirme ortamını hazır sunmak	Öğrencinin bilgisayarında çalışmak
Docker Compose	Uygulama bileşenlerini birlikte yönetmek	Web uygulaması ve veri servisini çalıştırmak

Docker Desktop kullanılan bilgisayarda bu araçları ayrı ayrı kurmak genellikle gerekmez. Linux sunucusunda ise Docker Engine ve Compose eklentisi kurulabilir.

3.2. Yerel ve bulut çalışma seçenekleri

Bu derste temel çalışma ortamı, Windows bilgisayarında Docker Desktop kullanarak Linux container'ları çalıştırmaktır. Docker komutları PowerShell üzerinden yürütülür. İmaj oluşturma, container yönetimi, ağ bağlantıları ve veri kalıcılığı işlemleri yerel ortamda gerçekleştirilebilir; bunlar için bulut hesabı veya kredi kartı gerekmez.

Docker Desktop, Windows Subsystem for Linux 2 (Linux için Windows Alt Sistemi, WSL 2) aracılığıyla gerekli Linux çalışma ortamını sağlar. Böylece Windows kullanılmaya devam edilirken Linux container'ları çalıştırılabilir [8], [9]. Kurulum, hazırlık, doğrulama ve sorun giderme adımları Ek 1 Windows Kurulum ve Hazırlık Kılavuzu'nda verilmiştir.

Alternatif çalışma ortamları Tablo 3'te özetlenmiştir.

Tablo 3. Yerel ve bulut ortamında container çalıştırma seçenekleri.

Ortam	Kullanılabilecek araçlar	Çalışma biçimi
Windows bilgisayar	Docker Desktop, WSL 2 ve PowerShell	Yerel çalışma
Linux bilgisayar	Docker Engine ve Compose eklentisi	Yerel çalışma
macOS bilgisayar	Docker Desktop	Yerel çalışma
Bulut Linux VM'si	Docker Engine ve Compose eklentisi	Uzak sunucuda çalışma

Linux ve macOS üzerinde de işletim sistemine uygun araçlarla aynı temel işlemler gerçekleştirilebilir [10], [11]. Dosya yolları ve terminal sözdizimi kullanılan ortama göre farklılaşabilir.

Bulut ortamında Docker, örneğin bir Linux VM'si üzerinde çalıştırılabilir. Bu durumda SSH ile sunucuya bağlanılır ve komutlar uzak sunucuda yürütülür. Uyumlu imaj yeniden kullanılabilir; ağ erişimi, yapılandırma ve kalıcı veri alanları hedef ortama göre düzenlenir.

Asgari gereksinim, yerel ortamda imaj oluşturabilmek ve container çalıştırabilmektir. Bulut erişimi, bu işlemleri öğrenmek için zorunlu değildir.

4. İmajlar ve Container Yaşam Döngüsü

4.1. İmaj, Container ve Katman İlişkisi

Bir uygulamanın paketlenmesi ile çalıştırılması farklı işlemlerdir. İmaj, uygulamayı çalıştırmak için gereken dosyaları ve varsayılan yapılandırmayı içeren pakettir. Container ise bu imajdan oluşturulan, kendi çalışma durumu ve kaynakları bulunan örnektir. Aynı imajdan birden fazla container oluşturulabilir; bunların süreçleri, ayarları ve çalışma sırasında ürettikleri veriler farklı olabilir [2], [12].

İmajın içeriği ve katmanlı yapısı:

Bir web uygulamasının imajı; uygulama kodunu, gerekli kütüphaneleri, çalışma ortamını ve başlangıç komutuna ilişkin bilgileri içerebilir. İmajın içinde uygulamanın bulunması, uygulamanın çalıştığı anlamına gelmez. Çalıştırma, imajdan bir container oluşturulup uygulama süreci başlatıldığında gerçekleşir [2].

İmajlar, layer (katman) adı verilen dosya sistemi değişikliklerinden oluşur. Bir katman dosya ekleyebilir, değiştirebilir veya silebilir. Örneğin bir web uygulaması imajı hazırlanırken temel çalışma ortamının üzerine bağımlılıklar ve uygulama dosyaları eklenebilir. Bu katmanlar birlikte container'ın kullanacağı dosya sistemi görünümünü oluşturur. Başlangıç komutu gibi yapılandırma bilgileri de imajın parçasıdır; ancak her yapılandırma işlemi yeni bir dosya sistemi katmanı oluşturmaz [2], [12].

İmaj katmanları ve container'ın yazılabilir katmanı:

İmaj, uygulamanın dosyalarını ve gerekli çalışma ortamını içeren katmanlı bir pakettir. Örneğin bir Python web uygulamasının imajında Python çalışma ortamı, kullanılan kütüphaneler, `app.py` uygulama dosyası ve `ayarlar.json` yapılandırma dosyası bulunabilir. Bu dosyalar imajdan ayrı nesneler değil, imajın içeriğidir. Katmanlar birlikte container'ın kullanacağı dosya sistemi görünümünü oluşturur [2].

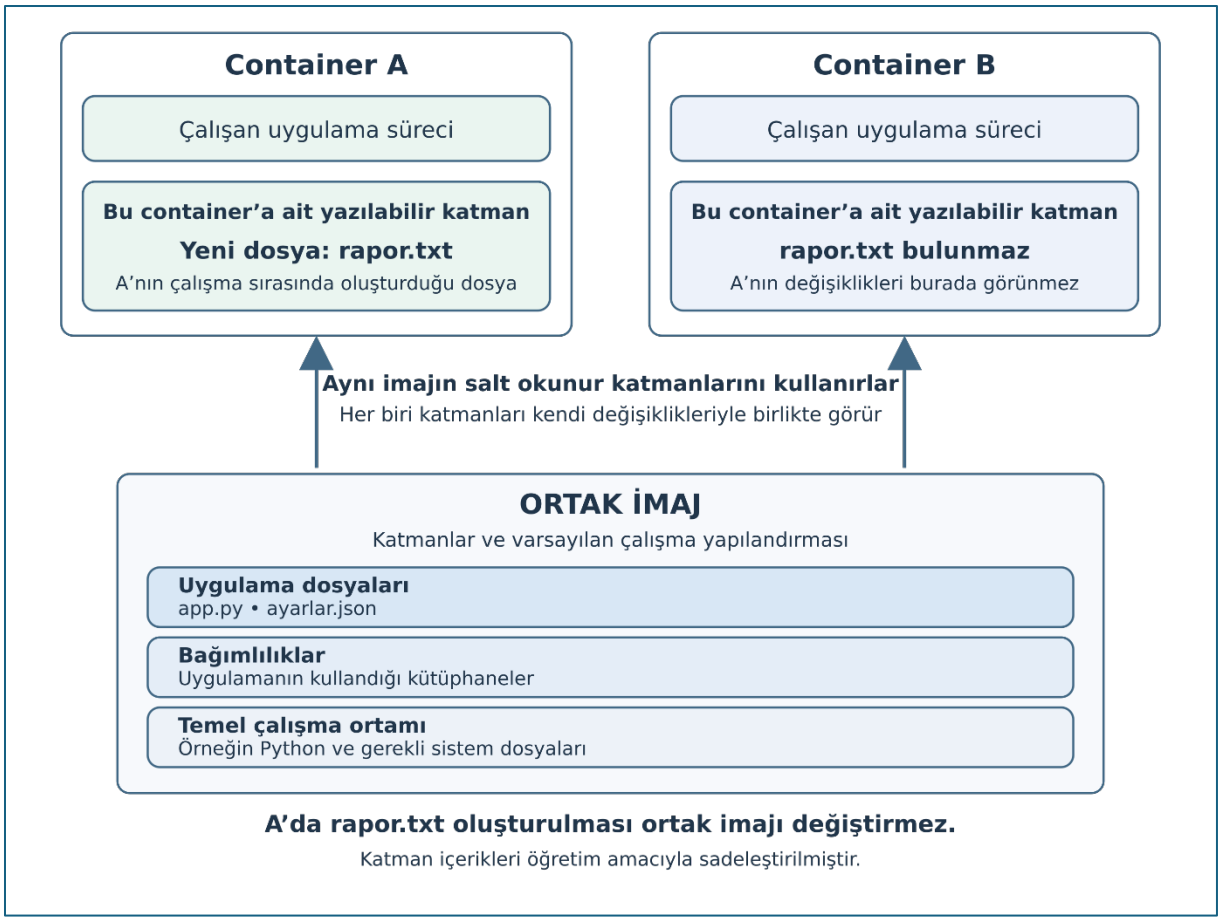
Container oluşturulduğunda imajın salt okunur katmanlarının üzerine o container'a ait bir yazılabilir katman eklenir. Uygulama çalışırken ayrı bir depolama alanına yönlendirilmeyen dosya değişiklikleri burada tutulur. Böylece container dosya oluşturabilir veya değiştirebilir; ancak kaynak imajın katmanlarını değiştirmez [12].

Şekil 1'de aynı imajdan oluşturulan iki container gösterilmektedir. Ortak imajın katmanları bir kez çizilmiş, her container'ın çalışma süreci ve yazılabilir katmanı ayrı gösterilmiştir.

Başlangıçta her iki container da imajdaki `app.py` ve `ayarlar.json` dosyalarını kullanır. Container A çalışırken `rapor.txt` adlı yeni bir dosya oluştururs a bu dosya A'nın yazılabilir katmanında tutulur. Dosya kaynak imaja eklenmez ve Container B'de kendiliğinden görünmez.

Mevcut bir dosyanın değiştirilmesinde de aynı ayrım geçerlidir. Örneğin A, `ayarlar.json` dosyasını değiştirirse değişiklik A'nın yazılabilir katmanına yansır. A dosyayı kendi değişikliğiyle görürken B, imajdaki özgün içeriği görmeye devam eder. Bu açıklama, dosyanın ortak bir depolama alanından bağlanmadığı durum için geçerlidir [12].

Container durdurulduğunda yazılabilir katmanı korunur; aynı container yeniden başlatıldığında dosyalar kullanılabilir. Container silindiğinde ise bu katman da kaldırılır. Aynı imajdan yeni bir container oluşturmak, silinen container'ın dosyalarını geri getirmez. Bu nedenle kalıcı veriler için Bölüm 6'da açıklanan depolama yöntemleri kullanılır [12].



Şekil 1. Aynı imajın salt okunur katmanlarını kullanan container'ların ayrı yazılabilir katmanları.

Uygulamayı güncellemenin anlamı:

İmaj, oluşturulduktan sonra değişmez bir paket olarak değerlendirilir. Uygulamanın yeni sürümünü dağıtmak için güncellenmiş bir imaj hazırlanır ve bundan yeni container'lar oluşturulur. Çalışan container içinde yapılan bir dosya değişikliği, kaynak imajı veya aynı imajdan oluşturulan diğer container'ları güncellemez [2], [12].

Bu ayrımın temel sonucu şudur: İmaj uygulamanın dağıtılabilir paketini, container uygulamanın çalışma örneğini, yazılabilir katman ise o örneğe ait dosya değişikliklerini temsil eder. Yerelde veya bulutta çalışırken bu ilişki aynıdır.

4.2. Registry, Repository, Tag ve Digest

İmajlar bilgisayarda yerel olarak saklanabilir. Ancak aynı imajı başka bir bilgisayarda veya bulutta kullanmak için imajın paylaşılabileceği bir hizmete ihtiyaç duyulur. Registry (imaj kayıt hizmeti) bu amaçla imajları saklar ve erişime sunar. Repository (imaj deposu) ise registry içinde belirli bir uygulamaya ait imajların toplandığı mantıksal depodur [13].

Örneğin bir üniversitenin registry hizmetinde web uygulaması ve veri servisi için ayrı repository'ler bulunabilir. Web uygulamasının farklı sürümleri aynı repository altında tutulur. Registry hizmetin tamamını, repository ise bu hizmet içindeki belirli bir imaj

koleksiyonunu ifade eder. Repository burada kaynak kod deposu anlamında kullanılmamaktadır [13].

Bir imaj adresini okumak:

Aşağıdaki temsili imaj adresini inceleyelim. Bu adres bir komut değil, imajın nerede bulunduğunu ve hangi etiketle seçildiğini gösteren bir başvurudur:

`registry.example.com/ders/web-uygulamasi:1.0`

Tablo 4. Örnek imaj adresinin bileşenleri.

Bileşen	Örnekteki karşılığı	Anlamı
Registry adresi	registry.example.com	İmajın saklandığı hizmetin adresi
Namespace (ad alanı)	ders	Kullanıcı, kurum veya proje için kullanılan ad grubu
Repository adı	web-uygulamasi	İlgili uygulamanın imaj deposu
Tag (etiket)	1.0	Depodaki bir imaja verilen okunabilir ad

Bu örnekte repository'nin registry içindeki tam yolu `ders/web-uygulamasi` olur. `1.0` etiketi, bu repository içindeki bir imajı seçmek için kullanılır. Docker'da registry adresi belirtilmezse varsayılan olarak Docker Hub kullanılır; tag belirtilmezse `latest` etiketi esas alınır [14].

Tag - okunabilir, ancak değişebilir bir ad:

Tag, imaja verilen bir etikettir. `1.0`, `2.0`, `test` veya `latest` gibi adlar kullanılabilir. Bir imajın birden fazla etiketi bulunabilir; örneğin `1.0` ve `latest` aynı imaja işaret edebilir [14].

Etiketin sürüm numarası gibi görünmesi, değişmez olduğu anlamına gelmez. Registry'nin politikası izin veriyorsa aynı etiket daha sonra başka bir imaja yönlendirilebilir. Örneğin bugün `latest` uygulamanın 1.0 sürümünü, yarın 2.0 sürümünü gösterebilir. `latest`, otomatik olarak en yeni sürümü bulan bir mekanizma değildir; yayıncının yönettiği bir etikettir [14], [15].

Digest - içeriğe bağlı tanımlayıcı:

Digest (kriptografik içerik özeti), imajı içeriğine bağlı bir değerle tanımlar. Yaygın gösterimi `sha256:` ile başlayan uzun bir değerdir. SHA-256, *Secure Hash Algorithm 256-bit* (256 bit Güvenli Özet Algoritması) anlamına gelir.

Teknik olarak digest, imajın katmanlarına ve yapılandırmasına başvuran tanım belgesinin özetini tanımlayabilir. Bu ayrıntının başlangıç düzeyindeki karşılığı şudur: Tag değişebilir bir ad sunarken digest belirli bir imaj içeriğini sabit olarak seçmeye yarar [15].

Digest ile başvuru biçimi şöyledir:

`registry.example.com/ders/web-uygulamasi@sha256:...`

Buradaki üç nokta, uzun özet değerinin metinde kısaltıldığını gösterir; gerçek kullanımda tam değer gerekir. Etiket başvurusunda `:` kullanılırken digest başvurusunda `@` kullanıldığına dikkat edilmelidir.

Örneğin yerelde doğrulanan imajın etiketi buluta dağıtım öncesinde başka bir imaja yönlendirilirse, aynı etiketi kullanmak aynı içeriğin seçildiğini garanti etmez. Digest ile başvuru bu belirsizliği azaltır. Bununla birlikte aynı imajın seçilmesi; ağ, veri ve çalışma ayarlarının da aynı olduğu anlamına gelmez.

4.3. Oluşturma, Çalıştırma, İnceleme, Durdurma ve Silme

Bir container'ın yaşam döngüsü, imajdan oluşturulmasıyla başlar. Container oluşturulduktan sonra başlatılabilir, çalışması incelenebilir, durdurulabilir ve kaldırılabilir. Oluşturulmuş bir container'ın mutlaka çalışıyor olması gerekmez; durmuş bir container da henüz silinmiş değildir [16], [17].

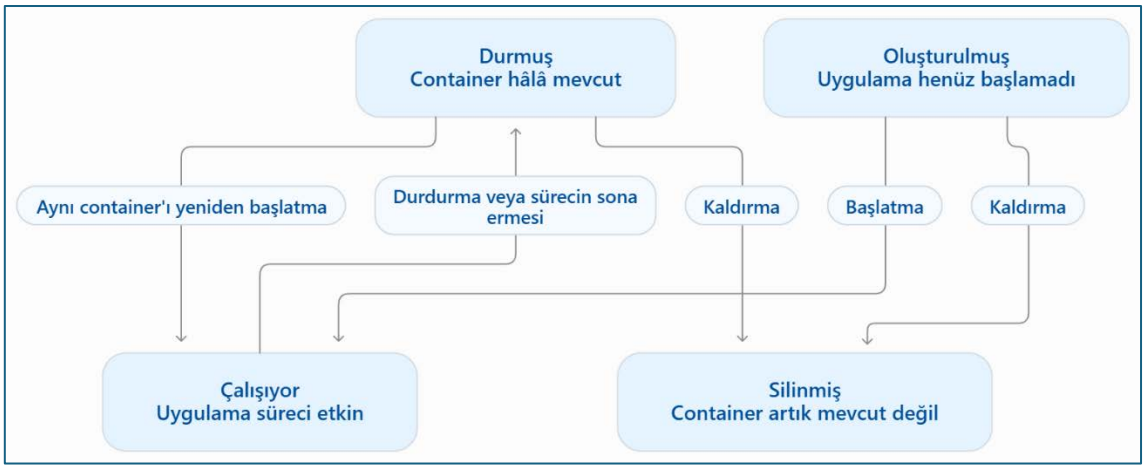
Oluşturma ve çalıştırma:

Oluşturma, seçilen imajdan bir container örneğinin hazırlanmasıdır. Bu aşamada container'ın kimliği, yazılabilir katmanı ve çalışma yapılandırması belirlenir. Başlangıç komutu, ağ bağlantıları ve veri bağlantıları bu yapılandırmanın parçalarıdır. Oluşturma işlemi tek başına uygulama sürecini başlatmaz [17].

Başlatma, oluşturulmuş container'ın uygulama sürecini çalıştırır. Docker'da oluşturma ve başlatma ayrı işlemler olarak yapılabileceği gibi tek bir işlemle de gerçekleştirilebilir. Burada önemli ayrım, yeni bir container oluşturmak ile mevcut bir container'ı yeniden başlatmak arasındadır. Aynı imajdan tekrar container oluşturmak, önceki örneği devam ettirmek anlamına gelmez [16], [17].

Container'ın çalışması, ana uygulama sürecinin yaşam süresine bağlıdır. Bir web sunucusunun sürekli çalışması beklenirken, yalnızca bir hesaplama yapıp sonuç üreten uygulama görevini tamamlayarak kapanabilir. Dolayısıyla container'ın durmuş olması tek başına hata göstergesi değildir. Arka planda çalıştırma da uygulama sona erdiğinde container'ın çalışmaya devam etmesini sağlamaz [16].

Şekil 2, temel durum geçişlerini göstermektedir. Şekilde otomatik silme ve zorla kaldırma gibi özel seçenekler gösterilmemiştir.



Şekil 2. Container'ın temel yaşam döngüsü ve durum geçişleri.

Çalışmayı inceleme:

İnceleme, container'ın beklenen işi yapıp yapmadığını anlamaya yöneliktir. Durum bilgisi ve log (günlük) kayıtları birlikte değerlendirilir. Durum bilgisi sürecin çalıştığını gösterebilir; günlükler ise başlangıç sırasında oluşan hataları veya uygulamanın ürettiği mesajları açıklayabilir [16].

Örneğin bir web container'ının çalışıyor görünmesi, web sayfasının kullanıcıya başarıyla sunulduğunu tek başına kanıtlamaz. Tarayıcıdan erişim de doğrulanmalıdır. İnceleme sırasında imaj adı ile container kimliği karıştırılmamalıdır: Aynı imajdan oluşturulmuş örneklerden hangisinin incelendiği, container adı veya kimliğiyle belirlenir [16].

Durdurma:

Durdurma işlemi, uygulamaya kapanması için bir sinyal gönderir. Uygulamanın işlemlerini tamamlaması için tanınan süre sonunda süreç hâlâ kapanmamışsa zorla sonlandırılır [18]. Bu nedenle uygulamanın kapanma isteğini doğru işlemesi, devam eden işlemlerin tamamlanması açısından önemlidir.

Normal durumda durdurma, container'ı silmez. Yazılabilir katman korunur; aynı container yeniden başlatılabilir. Önceki bölümdeki Container A durdurulup yeniden başlatılırsa, yazılabilir katmanındaki `rapor.txt` dosyası yerinde kalır [12].

Silme:

Silme, container örneğinin kaldırılmasıdır. Container'ın yazılabilir katmanı da kaldırılır; fakat kaynak imaj silinmez. Aynı imaj kullanılarak yeni bir container oluşturulabilir [12], [19].

Örneğin Container A silinip aynı imajdan yeni bir örnek oluşturulursa, A'nın yazılabilir katmanında üretilmiş `rapor.txt` yeni örnekte bulunmaz. Container dışındaki kalıcı depolamada tutulan verilerin yaşam döngüsü ise ayrıdır; bu konu Bölüm 6'da açıklanmaktadır.

5. Dockerfile ile Uygulama İmajı Oluşturma

5.1. Dockerfile'ın Görevi ve Temel Yönergeler

Bir uygulamayı imaja dönüştürmek için yalnız uygulama dosyalarını kopyalamak yeterli değildir. Uygulamanın hangi çalışma ortamına ihtiyaç duyduğu, bağımlılıklarının nasıl kurulacağı ve başlatıldığında hangi programın çalışacağı da tanımlanmalıdır.

Dockerfile, bu hazırlığı yönergelerle açıklayan bir metin dosyasıdır. Docker, imaj oluştururken bu yönergeleri işler. Varsayılan dosya adı uzantısız olarak Dockerfile şeklindedir. Dockerfile hazırlama tarifidir; imaj ise bu tarifin işlenmesiyle elde edilen pakettir [20].

Hazır uygulamadan imaja geçiş:

Örneğin hazır bir Python web uygulamasını paketlediğimizi düşünelim. Uygulamanın kaynak kodu ve bağımlılık listesi elimizde bulunmaktadır. Dockerfile ile şu hazırlıklar tanımlanır:

- Python içeren bir base image (temel imaj) seçilir.
- İmaj içindeki çalışma dizini belirlenir.
- Bağımlılık listesi kopyalanır ve gerekli kütüphaneler kurulur.
- Uygulama dosyaları kopyalanır.
- Container başlatıldığında uygulamanın nasıl çalıştırılacağı belirtilir.

Temel imaj seçmek, gerekli ortamı sıfırdan hazırlamak yerine mevcut bir imajın üzerine uygulamaya özgü bileşenleri eklemeyi sağlar. Dosyaların kopyalanması ve bağımlılıkların kurulması imajın içeriğini oluştururken, başlangıç komutu imajın çalışma yapılandırmasına kaydedilir [20].

Temel Dockerfile yönergeleri:

Tablo 5, başlangıç düzeyinde bilinmesi gereken yönergelerin görevlerini göstermektedir [21].

Tablo 5. Temel Dockerfile yönergeleri ve görevleri.

Yönerge	Görevi	Web uygulaması açısından anlamı
FROM	Temel imajı seçer.	Python çalışma ortamını sağlayan imajla başlamak
WORKDIR	Sonraki ilgili işlemler için çalışma dizinini belirler.	Uygulama dosyaları ve komutları için ortak bir konum kullanmak
COPY	Seçilen dosyaları imajın dosya sistemine kopyalar.	Bağımlılık listesini ve uygulama kodunu pakete eklemek

RUN	İmaj oluşturulurken komut çalıştırır.	Gerekli kütüphaneleri kurmak
CMD	Container başlatıldığında kullanılacak varsayılan komutu tanımlar.	Web uygulamasını başlatmak
EXPOSE	Uygulamanın kullanması beklenen container portunu belirtir.	Web servisinin dinlediği portu belgelemek

İmaj oluşturma ile uygulamayı çalıştırma ayrımı:

RUN ve CMD farklı zamanlarda görev yapar. RUN, imaj hazırlanırken; CMD ile tanımlanan komut ise container başlatılırken çalışır. Örneğin kütüphane kurulumu imaj hazırlığının parçasıdır; web sunucusunun başlatılması çalışma aşamasına aittir. Bu nedenle imaj oluşturmak, web uygulamasını kullanıcıların erişimine açmak anlamına gelmez [21].

EXPOSE da tek başına bilgisayarın veya bulut sunucusunun portunu erişime açmaz. Container portunun dışarıya yayımlanması, container çalıştırılırken ayrıca yapılandırılır. Bu ilişki Bölüm 6'da ele alınmaktadır [21].

Windows üzerinde çalışmak da bu ayrımı değiştirmez. Linux imajı hazırlanırken RUN işlemleri Linux derleme ortamında yürütülür; Dockerfile içindeki komutlar Windows PowerShell komutları olarak değerlendirilmez.

Yönergelerin sırası önemlidir: Bağımlılıkları kuracak araçların ilgili aşamada mevcut olması, kullanılacak dosyaların ise önceden kopyalanmış olması gerekir. Dosyaların imaj oluşturma işlemine nasıl sunulduğu ve bu sıranın önbellekle ilişkisi Bölüm 5.2'de açıklanmaktadır.

5.2. Derleme Bağlamı ve İmaj Oluşturma Süreci

Dockerfile, imajın nasıl hazırlanacağını tanımlar; ancak uygulama dosyalarının da imaj oluşturma işlemine sunulması gerekir. Build context (derleme bağlamı), bu işlem sırasında erişilebilen dosyalar kümesidir. Bu bölümde bağlamın, bilgisayardaki bir proje klasöründen sağlandığı temel yaklaşım ele alınmaktadır [22].

Buradaki "derleme", imajın hazırlanması anlamındadır. Uygulama kodunun mutlaka makine koduna çevrilmesi gerekmez; örneğin Python uygulamasında çalışma ortamı, bağımlılıklar ve kaynak dosyalar bir imaj içinde paketlenebilir.

Derleme bağlamı ile imaj içeriği arasındaki fark:

Bir proje klasöründe uygulama kodu, bağımlılık listesi, açıklama dosyaları ve geliştirme sırasında oluşan geçici dosyalar bulunabilir. Bu klasör derleme bağlamı olarak seçildiğinde, dışlanmamış dosyalar imaj oluşturma işleminin erişimine sunulur. Bir dosyanın bağlamda bulunması, otomatik olarak imaja eklendiği anlamına gelmez. İmaja

hangi dosyaların kopyalanacağı Dockerfile'daki COPY gibi yönergelerle belirlenir [21], [22].

Tablo 6'daki örnek, bu ayrımı göstermektedir.

Tablo 6. Örnek proje dosyalarının derleme bağlamı ve imaj içeriği açısından değerlendirilmesi.

Dosya veya klasör	Görevi	Örnekteki yaklaşım
app.py	Uygulama kodu	İmaja kopyalanır.
requirements.txt	Python bağımlılık listesi	İmaja kopyalanarak kurulumda kullanılır.
README.md	Projeyi açıklayan belge	Bağlamda bulunabilir; imaja kopyalanması gerekmez.
.venv/	Yerel Python sanal ortamı	Bağlamdan dışlanır; bağımlılıklar imajın içinde kurulur.
__pycache__/	Yerelde oluşan Python önbellek dosyaları	Bağlamdan dışlanır.

Yerel bağlamdan dosya kopyalarken kaynak yollar bağlamın köküne göre değerlendirilir. Bilgisayarda bir dosyanın bulunması yeterli değildir; dosyanın seçilen bağlam içinde erişilebilir olması gerekir. Dockerfile'ın konumu ile bağlamın kökü de aynı kavram değildir [22].

.dockerignore dosyasının görevi:

.dockerignore, bağlamdan dışlanacak dosya ve klasörleri tanımlar. Temel kullanımda bağlamın kökünde bulunur. Geçici dosyaların, yerel geliştirme ortamlarının ve gereksiz büyük dosyaların dışlanması, derlemeye sunulan içeriği azaltır ve yanlışlıkla imaja kopyalanmalarını önler [22].

Dışlanan bir dosya normal COPY işlemiyle imaja alınamaz. Bu nedenle uygulamanın ihtiyaç duyduğu dosyaları yanlışlıkla dışlamak, imaj oluşturmanın başarısız olmasına yol açabilir. .dockerignore, imaja kopyalanmış bir dosyayı sonradan temizleyen araç değildir [22].

Yönerge sırası ve önbellek:

Docker, uygun koşullarda önceki imaj oluşturma adımlarının sonuçlarını cache (önbellek) üzerinden yeniden kullanabilir. Bir adımın girdileri değiştiğinde o adımın ve ona bağlı sonraki adımların yeniden yapılması gerekebilir [23].

Örneğin önce bağımlılık listesi kopyalanıp kütüphaneler kurulursa, uygulama kodunun daha sonra kopyalanması yararlıdır. Yalnız app.py değiştiğinde, bağımlılık listesi ve önceki ilgili girdiler değişmemişse kütüphane kurulumu yeniden kullanılabilir. Bütün dosyaları

kurulumdan önce kopyalamak ise küçük bir kod değişikliğinin bağımlılık kurulumunu da tekrarlatmasına neden olabilir [23].

İşlem sonucunda uygulama dosyalarını ve çalışma yapılandırmasını içeren bir imaj elde edilir. İmajın başarıyla oluşturulması, uygulamanın doğru çalıştığını henüz kanıtlamaz; bunun için imajdan container oluşturularak çalışma davranışı doğrulanır.

5.3. İmajı Doğrulama ve Uygulamayı Güncelleme Yaklaşımı

İmaj oluşturma işleminin başarıyla tamamlanması, Dockerfile'daki hazırlık adımlarının sonuçlandığını gösterir. Ancak uygulamanın doğru başladığını, erişilebilir olduğunu veya beklenen sonucu ürettiğini henüz kanıtlamaz. Doğrulama, oluşturulan imajdan bir container çalıştırılarak uygulamanın davranışının sınanmasını gerektirir [16], [24].

Yerelde doğrulama:

Hazır web uygulaması için doğrulama üç düzeyde yapılabilir:

- Başlangıç: Container'ın uygulama süreci başlıyor mu? Günlüklerde eksik dosya, bulunamayan kütüphane veya yapılandırma hatası var mı?
- Erişim: Gerekli port bağlantısı sağlandığında uygulamaya Windows üzerindeki tarayıcıdan ulaşılabiliyor mu?
- İşlev: Sayfa beklenen içeriği gösteriyor mu? Uygulamanın sunduğu temel işlem doğru sonuç veriyor mu?

Örneğin container çalışıyor görünmesine rağmen tarayıcıdan erişilemiyorsa sorun port yapılandırmasında olabilir. Sayfa açılıyor fakat beklenen sonuç üretilmiyorsa uygulama davranışı incelenmelidir. Bu nedenle "container çalışıyor", "uygulamaya erişiliyor" ve "uygulama doğru çalışıyor" farklı doğrulama sonuçlarıdır. Durum bilgisi, günlükler ve kullanıcı tarafından görülen sonuç birlikte değerlendirilir [16].

Uygulama değiştiğinde ne olur?

Uygulama dosyaları imaja kopyalandıktan sonra, bilgisayardaki kaynak dosyanın değiştirilmesi mevcut imajı değiştirmez. Dosyaların dışarıdan bağlandığı geliştirme düzenleri dışında, yeni kodun paketlenmesi için imaj yeniden oluşturulmalıdır.

Örneğin `web-uygulamasi:1.0` imajındaki sayfaya yeni bir özellik eklendiğini düşünelim. Güncellenmiş kaynak dosyalardan `web-uygulamasi:1.1` imajı hazırlanır. Ardından bu imajdan yeni bir container oluşturularak özellik doğrulanır. Eski container, kendiliğinden yeni imajı kullanmaya başlamaz [2], [24].

Eski container'ı durdurup yeniden başlatmak da sürüm güncellemesi değildir. Yeniden başlatma aynı container'ı, oluşturulduğu imaj ve mevcut yapılandırmayla çalıştırır. Yeni imaja geçiş için container'ın yenilenmesi gerekir [16], [24].

Yeni sürüme geçiş ve verinin korunması:

Başlangıç düzeyindeki güncelleme yaklaşımı; yeni imajı oluşturmak, ayrı bir test container'ıyla doğrulamak ve ardından uygulamayı yeni imajdan oluşturulan container ile çalıştırmaktır. Eski ve yeni örnek aynı anda sınanacaksa container adları ve ana makinedeki portları çakışmamalıdır.

Container yenilenirken yazılabilir katmandaki verilerin yeni örneğe aktarılmadığı unutulmamalıdır. Uygulamanın ürettiği kalıcı veriler, imaj sürümünden ayrı yönetilmelidir. Veri koruma yöntemleri Bölüm 6'da açıklanmaktadır [12].

Önceki doğrulanmış imajın korunması, sorunlu bir güncellemede eski sürümden yeniden container oluşturmayı kolaylaştırır. Ancak uygulama verisinin biçimi de değişmişse yalnız eski imaja dönmek yeterli olmayabilir. Bu haftaki temel uygulamada güncelleme, veri yapısını değiştirmeyen küçük bir özellik veya sayfa içeriği değişikliği üzerinden ele alınır.

6. Container Verisi ve Ağ Bağlantıları

6.1. Container Dosya Sistemi ve Kalıcı Veri Yönetimi

Bir uygulamanın çalıştırılması için gereken dosyalar ile çalışırken ürettiği veriler farklı amaçlara hizmet eder. Uygulama kodu ve bağımlılıklar imajda bulunabilir; kullanıcı kayıtları, yüklenen dosyalar veya veri tabanı içeriği ise uygulamanın çalışması sırasında oluşur. Container yenilenirken uygulama paketi değişebilir; ancak gerekli verilerin korunması gerekir.

Yazılabilir katmanın sınırı:

Bölüm 4.1'de açıklandığı gibi, ayrı bir depolama alanına yönlendirilmeyen dosya değişiklikleri container'ın yazılabilir katmanında tutulur. Bu katman container durdurulduğunda korunur, container silindiğinde kaldırılır. Bu nedenle yalnızca container'ı durdurup başlatmak, verinin container silindikten sonra da korunacağını göstermez [12], [25].

Kalıcı veri yönetiminde amaç, gerekli verilerin yaşam süresini belirli bir container örneğinin yaşam süresinden ayırmaktır. Bunun için depolama alanı, container'ın dosya sistemindeki belirli bir dizine mount (bağlama) işlemiyle bağlanır. Uygulama bu dizine normal bir dosya yolu üzerinden erişir; veri ise yazılabilir katmanın dışında tutulur [25].

Volume: Docker tarafından yönetilen veri alanı:

Volume (kalıcı veri alanı), Docker tarafından oluşturulan ve yönetilen bir depolama alanıdır. Bu haftada özellikle named volume (adlandırılmış veri alanı) yaklaşımı kullanılır. Örneğin `uygulama-verisi` adlı volume, container içindeki `/veri` dizinine bağlanabilir. Uygulama verilerini bu dizine yazarsa veriler volume'da tutulur [26].

Container kaldırılıp yeni bir örnek oluşturulduğunda, aynı volume doğru dizine yeniden bağlanırsa yeni örnek mevcut verilere erişebilir. Yeni container oluşturmak, eski volume'u kendiliğinden seçmez. Volume'un ayrıca silinmesi de verilerin kaybına yol açar [26].

Bind mount - belirli bir klasörü container'a bağlama:

Bind mount (doğrudan dosya veya dizin bağlama), Docker'ın çalıştığı ana makinedeki belirli bir yolu container'a bağlar. Docker Desktop, Windows'taki klasörlerin Linux container'larına sunulmasını sağlar. Örneğin Windows'taki bir proje klasörü, container içinde `/uygulama` yolundan erişilebilir hâle getirilebilir [27].

Bu yöntem, dosyaları Windows üzerinde düzenleyip container'dan kullanmak için yararlıdır. Ancak imaja kopyalamadan farklıdır: Container, bağlanan klasörün mevcut içeriğini görür. Yazma izni varsa container'ın yaptığı değişiklikler ana makinedeki dosyalara da yansıyabilir; yalnız okuma gereken durumlarda bağlantı salt okunur yapılabilir [27].

Tablo 7, üç yaklaşımın temel farklarını özetlemektedir.

Tablo 7. Container verisinin tutulduğu alanların karşılaştırılması.

Yaklaşım	Veri nerede tutulur?	Container silindiğinde	Temel kullanım
Yazılabilir katman	Container'a ait katmanda	Bu katmandaki veri kaldırılır.	Container'a özgü geçici dosyalar
Adlandırılmış volume	Docker'ın yönettiği ayrı alanda	Volume ayrıca silinmedikçe korunur.	Uygulama veya veri servisinin kalıcı verileri
Bind mount	Ana makinedeki seçilen dosya veya klasörde	Kaynak dosyalar korunur.	Yerel dosyalarla çalışma ve geliştirme

Depolama alanını doğru yere bağlamak:

Bir volume oluşturmak tek başına yeterli değildir. Uygulamanın gerçekten veri yazdığı dizine bağlanmalıdır. Örneğin uygulama `/veri` yerine `/uygulama/kayitlar` dizinine yazıyorsa, `/veri` dizinine bağlanan volume bu kayıtları korumaz.

Ayrıca bind mount, bağlandığı dizinde imajdan gelen dosyaları görünmez hâle getirebilir. Örneğin uygulama kodunun bulunduğu dizine boş bir klasör bağlanırsa kod silinmiş olmasa da container o konumda bağlı klasörün içeriğini görür [27].

Son olarak kalıcılık, yedekleme değildir. Yerel volume veya klasörün container'dan bağımsız olması, disk arızasına ya da yanlışlıkla silinmeye karşı koruma sağladığı anlamına gelmez. Aynı imajın bulutta çalıştırılması da yereldeki verileri otomatik olarak buluta taşımaz.

6.2. Container Ağı, Port Yayımlama ve Servisler Arası Bağlantı

Bir web uygulamasında iki farklı bağlantı gereksinimi bulunabilir: Kullanıcının tarayıcıdan uygulamaya erişmesi ve uygulamanın veri servisine bağlanması. Bu bağlantılar farklı adresler ve portlar kullanabilir. Container ağı, uygulama bileşenlerinin birbirleriyle ve dış ortamla iletişimini sağlar [28].

Bu bölümde, Docker Desktop üzerinde çalışan Linux container'larının ayrı ağ ortamlarına sahip olduğu temel yaklaşım ele alınmaktadır. Her container kendi ağ arayüzünü ve bağlantı bilgilerini görür [28].

Container portu ve ana makine portu:

Container portu, uygulamanın container içinde dinlediği porttur. Ana makine portu ise Windows bilgisayarı üzerinden erişim için kullanılan porttur. Bu iki portun numarası aynı olmak zorunda değildir.

Örneğin web uygulaması container içinde `8000` portunda çalışıyor olsun. Windows bilgisayarının `8080` portu bu porta yönlendirilirse, tarayıcıdan `http://localhost:8080` adresine yapılan istek uygulamanın `8000` portuna ulaşır. Uygulamanın dinlediği port değişmez; dışarıdan kullanılan erişim noktası belirlenmiş olur.

Bu düzenleme port publishing (port yayımlama) olarak adlandırılır. Dockerfile'daki `EXPOSE`, portun kullanımını belgeler; bu yönlendirmeyi tek başına oluşturmaz [21], [29].

Yerel çalışma için ana makine tarafındaki bağlantı `127.0.0.1` adresiyle sınırlandırılabilir. Böylece yayımlanan porta kendi bilgisayarından erişilir. Ana makine adresi belirtilmeden yayımlanan portlar varsayılan olarak daha geniş erişime açılabileceğinden, yerel çalışma ile dışarıya hizmet sunma ayrı değerlendirilmelidir [29].

`localhost` hangi ortamı gösterir?

`localhost`, bağlantıyı başlatan ortamın kendisini ifade eder. Bu nedenle Windows tarayıcısında kullanılan `localhost` ile container içindeki uygulamanın kullandığı `localhost` aynı hedef değildir.

Örneğin web uygulaması ayrı bir veri tabanı container'ına bağlanacaksa, bağlantı adresi olarak `localhost` kullanması kendi container'ında veri tabanı aramasına neden olur. Veri tabanı başka bir container'da olduğundan, o servisin ağ üzerinden erişilebilir adı kullanılmalıdır [28].

Benzer şekilde, uygulamanın container içinde yalnız `127.0.0.1` üzerinde dinlemesi diğer ortamlardan gelen bağlantıları engelleyebilir. Uygulamanın `0.0.0.0` üzerinde dinlemesi, container'ın tüm IPv4 arayüzlerinden bağlantı kabul etmesini sağlar. Bu ayar, ana makinedeki portu yayımlama işleminin yerine geçmez.

Container'ların servis adıyla bağlantısı:

Aynı user-defined bridge network (kullanıcı tanımlı köprü ağı) üzerindeki container'lar, container adları veya ağ takma adları üzerinden birbirlerine erişebilir. Docker'ın ad çözümleme mekanizması bu adları ilgili ağ adreslerine çevirir. Bu özellik, varsayılan köprü ağındaki adla erişim davranışıyla karıştırılmamalıdır [30].

Örneğin veri servisine bu ağda `db` adı verilmiş ve servis `5432` portunda dinliyor olsun. Web uygulaması veri servisine `db:5432` üzerinden bağlanır. Bunun için veri servisinin portunu Windows bilgisayarına yayımlamak gerekmez [30].

Tablo 8, örnekteki iki bağlantının farkını göstermektedir.

Tablo 8. Tarayıcı erişimi ile servisler arası bağlantının karşılaştırılması.

Bağlantıyı başlatan	Hedef	Kullanılan adres	Gerekli düzenleme
Windows tarayıcısı	Web container'ı	`localhost:8080`	Ana makine portunun container'daki `8000` portuna yayımlanması
Web container'ı	Veri servisi	`db:5432`	Ortak kullanıcı tanımlı ağ ve `db` adının bu ağda tanımlanması

Container yeniden oluşturulduğunda ağ adresi değişebilir. Bu nedenle uygulama yapılandırmasında değişebilen adresleri sabitlemek yerine servis adlarını kullanmak daha uygundur [30]. Ancak ortak ağ ve doğru adres yalnız bağlantı yolunu sağlar; hedef servisin çalışıyor, bağlantı kabul ediyor ve gerekli kimlik bilgileriyle erişilebilir olması da gerekir.

6.3. Veri ve Bağlantı Sorunlarını Anlama

Container'ın çalışıyor görünmesi, uygulamanın verilerini doğru yerde sakladığını veya diğer servislere bağlanabildiğini tek başına göstermez. Sorunu anlamak için belirtiyi, ilgili yapılandırmayı ve uygulamanın günlüklerini birlikte değerlendirmek gerekir. İlk adım, sorunun veri saklama, ağ bağlantısı veya uygulama davranışıyla ilgili olduğunu ayırt etmektir.

Veri kaybı ile yanlış veri alanını ayırmak:

Container yenilendikten sonra kayıtların görünmemesi, verilerin mutlaka silindiği anlamına gelmez. Yeni container farklı veya boş bir volume'a bağlanmış olabilir. Aynı volume kullanılıyor olsa bile yanlış dizine bağlanması, uygulamanın başka bir konuma yazmasına neden olabilir [26].

Örneğin önceki container `/veri` dizinine bağlı `uygulama-verisi` volume'unu kullanırken yeni örnek başka bir volume'a bağlanmışsa, uygulama boş bir veri alanı görebilir. Sorunu

çözmek için önce hangi depolama alanının, hangi container dizinine bağlandığı kontrol edilmelidir.

Bind mount kullanılan bir dizinde imajdan gelen dosyaların görünmemesi de silinme olarak yorumlanmamalıdır. Bağlanan klasör, o konumdaki mevcut içeriği örter. Yazma hatalarında ise bağlantının salt okunur olması veya dosya erişim izinleri değerlendirilir [27].

Bağlantı yolunu aşamalı değerlendirmek:

Tarayıcıdan web uygulamasına erişilemiyorsa önce uygulamanın çalıştığı ve beklenen container portunda dinlediği doğrulanır. Ardından ana makine portunun doğru container portuna yönlendirilip yönlendirilmediğine bakılır. Yanlış port eşleştirmesi ile uygulamanın hiç başlamamış olması farklı sorunlardır [17], [29].

Web uygulaması açılıyor fakat veri servisine erişemiyorsa, tarayıcı bağlantısı sağlanmış demektir; servisler arası bağlantı ayrıca incelenmelidir. Ortak ağ, hedef servis adı, hedef port ve veri servisinin bağlantı kabul edecek durumda olması kontrol edilir. Web container'ından ayrı bir veri servisine bağlanırken `localhost` kullanmak, hedefi yanlış belirler [28], [30].

Tablo 9, sık görülen belirtiler ile olası nedenleri eşleştirmektedir. Bunlar kesin teşhis değil, incelemeyi yönlendiren başlangıç noktalarıdır.

Tablo 9. Veri ve bağlantı sorunlarında başlangıç kontrolleri.

Belirti	Olası neden	Önce ne incelenir?
Container yenilenince kayıtlar görünmüyor.	Veri yazılabilir katmanda kalmış veya farklı volume bağlanmış olabilir.	Önceki ve yeni depolama bağlantıları
Volume bağlı olduğu hâlde veriler korunmuyor.	Uygulama bağlanan dizinin dışında yazıyor olabilir.	Uygulamanın gerçek veri yolu
Klasör bağlandıktan sonra uygulama dosyaları görünmüyor.	Bind mount, imajdaki dosyaları örtmüş olabilir.	Kaynak klasör ve hedef dizin
Uygulama dosyaya yazamıyor.	Salt okunur bağlantı veya izin sorunu olabilir.	Bağlantı türü ve erişim izinleri
Container çalışıyor, web sayfası açılmıyor.	Dinleme adresi veya port eşleştirmesi yanlış olabilir.	Uygulama günlükleri ve iki taraftaki portlar
Web uygulaması veri servisine bağlanamıyor.	Yanlış ad, port, ağ veya henüz hazır olmayan servis söz konusu olabilir.	Bağlantı ayarları ve veri servisi günlükleri

Bulgulara göre müdahale etmek

Ağ üzerinden hedefe ulaşılması, kimlik doğrulamanın da başarılı olduğu anlamına gelmez. Kullanıcı adı veya parola hatası, ağ erişimi sorunundan ayrı değerlendirilmelidir.

Benzer şekilde servisin sürecinin başlaması ile bağlantı kabul etmeye hazır olması arasında süre bulunabilir.

İncelemede her seferinde ilgili bir ayar değiştirilip sonuç yeniden doğrulanmalıdır. Özellikle veri sorunlarında volume'ları silerek baştan başlamak, sorunun kanıtlarını ve gerekli verileri ortadan kaldırabilir. Önce mevcut bağlantıları ve günlükleri incelemek, sonra nedene uygun düzeltme yapmak temel yaklaşımdır.

7. Docker Compose ile Çok Bileşenli Uygulama Yönetimi

7.1. Compose'un Görevi ve Uygulama Bileşenlerinin Tanımlanması

Bir web uygulaması, web servisi ve veri tabanı gibi birden fazla bileşenden oluşabilir. Bunları ayrı container'larda çalıştırırken her birinin imajını, ağ bağlantılarını, veri alanlarını ve çalışma ayarlarını belirlemek gerekir. Docker Compose, bu bileşenleri ortak bir uygulama tanımı altında düzenlemeyi ve birlikte yönetmeyi sağlar [31].

Compose, yapılandırmayı `compose.yaml` adlı metin dosyasından okur. Dosya, YAML — YAML Ain't Markup Language (YAML bir işaretleme dili değildir) biçimini kullanır. Bu biçimde girintiler, alanların hangi bölüme ait olduğunu belirler. Dosya bir komut dizisi değil, uygulamanın bileşenlerini ve bunların yapılandırmasını açıklayan tanımdır [31], [32].

Servis, container ve proje ilişkisi:

Compose içinde service (servis), uygulamanın bir bileşeninin nasıl çalıştırılacağını tanımlar. Örneğin `web` servisi web uygulamasını, `db` servisi veri tabanını temsil edebilir.

Servis tanımı ile container aynı şey değildir. Servis tanımından çalışan container örnekleri oluşturulur. Compose, ilgili servisleri ve kaynakları bir project (proje) altında gruplar; böylece uygulama bir bütün olarak yönetilebilir [31].

Uygulama tanımında hangi bilgiler bulunur?

Tablo 10, temel Compose alanlarının görevlerini göstermektedir. Bunlar dosya içindeki alan adlarıdır; tek başlarına çalıştırılacak komutlar değildir [32].

Tablo 10. Temel Compose alanları ve uygulamadaki görevleri.

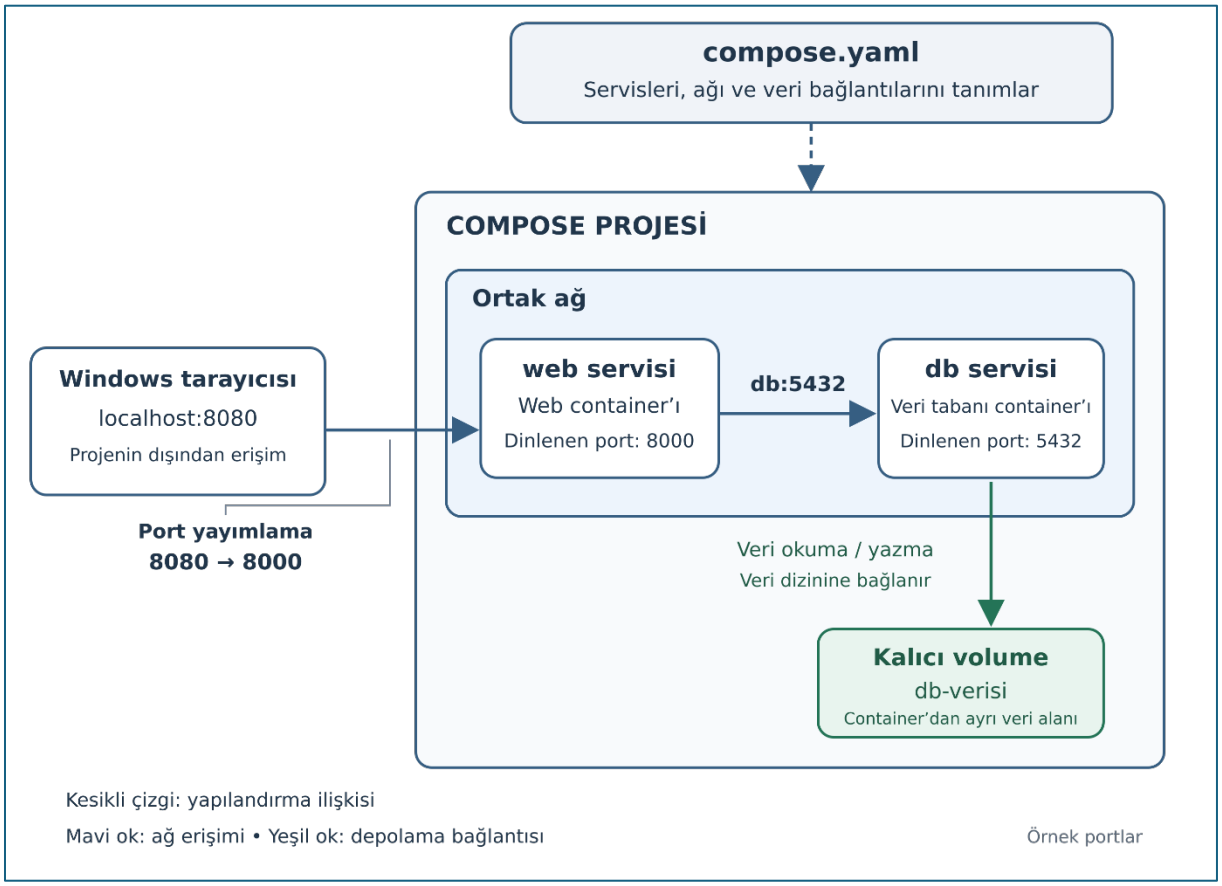
Alan	Görevi	Örnek kullanım
`services`	Uygulamanın servislerini tanımlar.	`web` ve `db` bileşenleri
`image`	Servisin kullanacağı imajı belirtir.	Hazır veri tabanı imajını seçmek
`build`	Servis imajının nasıl oluşturulacağını belirtir.	Web uygulamasını Dockerfile'dan paketlemek

ports	Ana makine ile container arasındaki port eşleştirmelerini tanımlar.	Tarayıcı erişimini sağlamak
environment	Container'a ortam değişkenleri sağlar.	Veri servisi adresini uygulamaya bildirmek
volumes	Veri bağlantılarını tanımlar.	Veri tabanı dizinini kalıcı alana bağlamak
networks	Servislerin bağlanacağı ağları tanımlar.	Bileşenlerin ortak ağda iletişimini sağlamak

Compose dosyası Dockerfile'ın yerine geçmez. Dockerfile imajın hazırlanmasını, Compose ise uygulama bileşenlerinin bu imajlarla nasıl çalıştırılacağını tanımlar. Hazır imaj kullanan bir servis için ayrıca Dockerfile yazılması gerekmez [32].

Web uygulaması ve veri servisi örneği:

Şekil 3'te web ve db servislerinden oluşturulan container'lar, ortak ağ ve kalıcı veri alanı gösterilmektedir. Özel bir ağ yapılandırması yapılmadığında Compose, proje için varsayılan bir ağ oluşturur ve servisleri bu ağa bağlar [33].



Şekil 3. Compose ile tanımlanan web uygulaması, veri servisi, ortak ağ ve kalıcı veri alanı.

Windows tarayıcısı, ana makinenin 8080 portu üzerinden web container'ının 8000 portuna erişir. Web uygulaması ise veri servisine ortak ağdaki db adı ve 5432 portuyla

bağlanır. Bu iç bağlantı için veri tabanı portunun Windows'a yayımlanması gerekmez. Veri servisinin kayıtları, container'dan ayrı volume'da tutulur [26], [33].

Compose'un db adlı servisi tanımlaması, web uygulamasının bağlantı ayarlarını kendiliğinden oluşturmaz. Veri tabanı adresi ve gerekli diğer bilgiler uygulamaya ayrıca verilmelidir. Uygulamanın beklediği ayarlar ile Compose dosyasında verilen değerler uyumlu olmalıdır.

7.2. Birlikte Başlatma, Bağımlılıklar ve Hazır Olma

Compose, uygulama tanımına göre gerekli container'ları, ağları ve veri bağlantılarını hazırlar; servisleri başlatır. Ancak servislerin birlikte yönetilmesi, aynı anda kullanıma hazır olacakları anlamına gelmez [31], [34].

Örneğin veri tabanı süreci başladıktan sonra başlangıç dosyalarını hazırlayabilir ve mevcut verileri kontrol edebilir. Bu sırada container çalışıyor görünse de veri tabanı henüz bağlantı kabul etmeyebilir. Web uygulaması erken bağlantı kurmaya çalışırsa hata alabilir [34].

Bağımlılık sırasının tanımlanması:

Web uygulamasının veri servisine ihtiyaç duyması, servisler arasında bir bağımlılık bulunduğunu gösterir. Compose dosyasındaki depends_on alanı bu ilişkiyi tanımlamak için kullanılır.

Örneğin web servisinin db servisine bağlı olduğu belirtildiğinde Compose, başlatma işlemlerini bu ilişkiye göre düzenler. Ancak yalnızca başlatma sırası tanımlamak, veri servisinin bağlantı kabul etmeye hazır olduğunu garanti etmez. Hazır olmayı beklemek için ayrıca uygun bir koşul gerekir [32], [34].

Sağlık kontrolüyle hazır olmayı değerlendirmek:

Health check (sağlık kontrolü), servisin belirlenen bir ölçüte göre çalışabilir durumda olup olmadığını sınayan kontroldür. Compose içinde healthcheck alanıyla tanımlanabilir. Veri tabanı için bağlantı kabul etme durumunu, web uygulaması için belirli bir adresin beklenen yanıtı üretmesini kontrol edebilir.

Kontrolün ne zaman ve ne sıklıkta yapılacağı, ne kadar bekleneceği ve başarısız sonuçların nasıl değerlendirileceği yapılandırılır. Container'ın sağlık durumu başlangıçta starting, kontrollerin sonucuna göre healthy veya unhealthy olabilir [32].

Bağımlılık için service_healthy koşulu kullanıldığında Compose, bağımlı servisi başlatmadan önce hedef servisin sağlık kontrolünü geçmesini bekler. Örnekte veri tabanının uygun kontrolden geçmesi, web uygulamasının başlatılması için koşul hâline gelir [34].

Sağlıklı görünmek ile doğru çalışmak arasındaki fark:

Sağlık kontrolünün sonucu, yalnızca tanımlanan kontrolün kapsamı kadar bilgi verir. Bir web sayfasının yanıt üretmesi, uygulamanın bütün özelliklerinin doğru çalıştığını göstermez. Tablo 11 bu ayrımı özetlemektedir.

Tablo 11. Servislerin çalışma ve doğrulama düzeyleri.

Düzey	Ne gösterir?	Tek başına neyi göstermez?
Container çalışıyor	Ana süreç henüz sona ermemiştir	Servisin bağlantı kabul ettiğini
Sağlık kontrolü başarılı	Tanımlanan kontrol geçilmiştir	Uygulamanın bütün işlevlerinin doğru olduğunu
Uygulama işlemi doğrulanmış	Sınanan işlem beklenen sonucu vermiştir	Sınanmayan bütün durumların doğru olduğunu

Örneğin veri tabanı bağlantı kabul etse bile web uygulamasının kullanıcı adı veya parolası yanlış olabilir. Bu nedenle son doğrulama, web uygulaması üzerinden veri servisini kullanan bir işlemin başarıyla tamamlanmasını da içermelidir.

Başlangıçta hazır olmayı beklemek, çalışma sırasında oluşabilecek bağlantı kesintilerini çözmez. Sağlık durumunun unhealthy olması da tek başına otomatik yeniden başlatma anlamına gelmez. Uygulamanın bağlantı hatalarını anlaşılır biçimde kaydetmesi ve gerektiğinde yeniden bağlantı denemesi yapması ayrı bir çalışma davranışıdır [32].

7.3. Uygulamayı İnceleme, Güncelleme ve Veriyi Koruyarak Sonlandırma

Compose ile yönetilen bir uygulamada yalnızca tek bir container'ın değil, servislerin birlikte çalışma durumunun değerlendirilmesi gerekir. Web servisi çalışırken veri servisi durmuş olabilir; iki servis çalışıyor görünse de aralarındaki bağlantı başarısız olabilir. Bu nedenle inceleme, servis durumları, sağlık kontrolleri, günlükler ve uygulamanın ürettiği sonuç üzerinden yapılır [31].

Uygulamayı servis düzeyinde inceleme:

Günlükleri hangi servisin ürettiği dikkate alınmalıdır. Örneğin web servisindeki "veri tabanına bağlanılamadı" mesajı sorunun belirtisidir; asıl neden veri servisinin başlangıç günlüklerinde bulunabilir.

İncelemede önce hangi servisin beklendiği gibi çalışmadığı belirlenir. Ardından o servisin imajı, çalışma ayarları, ağ ve veri bağlantıları değerlendirilir. Böylece bütün uygulamayı kaldırıp yeniden oluşturmak yerine sorunun ilgili bileşenine odaklanılır.

İmaj ve yapılandırma değişikliklerini uygulama:

Uygulama kodu imaja kopyalanıyorsa, kod değişikliğinden sonra yeni imaj oluşturulmalı veya hazır yeni sürüm kullanılmalıdır. Compose, servis imajı ya da yapılandırması

değiştiğinde ilgili container'ları yeniden oluşturarak değişiklikleri uygulayabilir; bağlı volume'lar korunur [35].

Ancak yalnızca yeniden başlatmak, yeni imajı veya değiştirilmiş Compose ayarlarını uygulamak değildir. Örneğin compose.yaml içindeki ortam değişkeni değiştirilmişse mevcut container'ın yeniden başlatılması bu değeri güncellemez. Yeni yapılandırmayla container'ın yeniden oluşturulması gerekir [36].

Güncelleme sonrasında servislerin hazır olması ve uygulamanın temel işlemleri tekrar doğrulanır. Veri servisi aynı veri alanını kullanacaksa volume bağlantısının korunması gerekir. Volume tanımı veya proje adı değiştiğinde farklı bir veri alanının kullanılabileceği de dikkate alınmalıdır [26], [31].

Durdurma ile kaldırma arasındaki fark:

Uygulamaya geçici olarak ara vermek ile çalışma ortamını kaldırmak farklı işlemlerdir. Tablo 12, temel ayrımı göstermektedir.

Tablo 12. Compose uygulamasını sonlandırma seçeneklerinin kaynaklara etkisi.

İşlem	Container'lar	Proje ağı	Adlandırılmış volume'lar
Servisleri durdurma	Korunur; süreçleri durur.	Korunur.	Korunur.
Varsayılan kaldırma	Kaldırılır; yazılabilir katmanları silinir.	Compose'un yönettiği ağlar kaldırılır.	Korunur.
Volume'ları da kaldırma	Kaldırılır.	Compose'un yönettiği ağlar kaldırılır.	Kaldırma kapsamındaki volume'lar ve verileri silinir.

Durdurulan container'lar tekrar başlatılabilir. Varsayılan kaldırma ise container'ları ve ilgili ağları kaldırır; imajları ve adlandırılmış volume'ları varsayılan olarak silmez. Dış kaynak olarak tanımlanan ağlar ve volume'lar Compose tarafından kaldırılmaz [37], [38].

Veriyi koruyarak çalışmayı bitirme:

Örneğin veri tabanına kayıt eklenmişse, container'lar kaldırıldıktan sonra da bu kayıtların korunması istenebilir. Bunun için verilerin doğru volume'da bulunduğu doğrulanmalı ve sonlandırmada volume silme seçeneği kullanılmamalıdır. Uygulama yeniden oluşturulduğunda aynı veri alanı bağlanırsa kayıtlar tekrar kullanılabilir [26], [38].

İmajı saklamak, uygulama verisini saklamak değildir. İmaj uygulama paketini içerir; çalışma sırasında oluşan kayıtlar ayrı depolamada tutulur. Bu nedenle temizleme işleminin kapsamı, "çalışan bileşenleri kapatmak" ile "verileri de silmek" arasında bilinçli olarak seçilmelidir.

8. Container İmajlarının Bulutta Kullanılması

Yerelde oluşturulan ve doğrulanan bir imaj, uygun bir bulut çalışma ortamında da kullanılabilir. Temel yaklaşım, uygulamayı her ortamda yeniden kurmak yerine aynı uygulama paketini farklı ortamlarda çalıştırmaktır. Ancak imajın taşınması; ağ ayarlarının, kalıcı verilerin ve erişim izinlerinin de taşındığı anlamına gelmez. Bu bölüm, yerel çalışmadan buluta geçişin mantığını açıklamaktadır; kurulum ve dağıtım adımları ek kılavuzda ele alınır.

8.1. İmajı Registry Üzerinden Yerelden Buluta Taşıma

Bölüm 4.2'de açıklanan registry, imajın oluşturulduğu bilgisayar ile kullanılacağı ortam arasında paylaşım noktasıdır. Yerelde hazırlanan imaj registry'ye yüklenir; buluttaki çalışma ortamı aynı imajı buradan edinir [13], [39], [40].

Bu süreçte iki temel işlem bulunur:

- Push (yükleme): Yerelde bulunan imajın registry'ye gönderilmesi.
- Pull (indirme): Registry'de bulunan imajın hedef çalışma ortamına alınması.

Registry'ye imaj yüklemek, uygulamayı bulutta çalıştırmak değildir. Yükleme yalnız paketi erişilebilir hâle getirir. Uygulamanın çalışması için buluttaki ortamın bu imajı kullanarak container oluşturması ve başlatması gerekir [39], [40].

İmajın adı ve erişim yetkileri:

İmajın registry adresi, repository yolu ve etiketi, hangi paketin paylaşılacağını belirtir. Örneğin yerelde doğrulanan web uygulaması, temsili olarak şu başvuruyla yayımlanabilir:

```
registry.example.com/ders/web-uygulamasi:1.0
```

Bölüm 4.2'deki adres yapısı burada da geçerlidir. Bulut ortamı, bu başvuruyu kullanarak ilgili imajı edinir. Registry'ye yükleme için gerekli yetki ile imajı indirme yetkisi farklı olabilir. Özel bir repository kullanılıyorsa hedef ortamın da imajı okumaya yetkili olması gerekir [39], [40].

Registry'nin uygulamayla aynı makinede bulunması gerekmez. Buradaki görevi uygulamayı çalıştırmak değil, imajı saklamak ve yetkili ortamlara sunmaktır [13].

Bulutta aynı içeriği seçmek:

Yerelde test edilen imajın bulutta da kullanıldığını doğrulamak için yalnız etiket adına bakmak yeterli olmayabilir. Etiket değiştirilebildiğinden, daha sonra aynı adla indirilen imaj farklı içerik taşıyabilir. Belirli içeriğe bağlı kalmak gerektiğinde digest üzerinden başvuru yapılabilir [15], [40].

Örneğin `1.0` etiketiyle yayımlanan imajın digest değeri kaydedilirse, hedef ortamda o değerle belirtilen içerik seçilebilir. Böylece yerelde doğrulanan paket ile bulutta kullanılan paket arasındaki belirsizlik azaltılır [40].

İmajla birlikte ne taşınmaz?

İmaj; içine paketlenmiş uygulama dosyalarını, bağımlılıkları ve varsayılan çalışma yapılandırmasını içerir. Buna karşılık yerel container'ın yazılabilir katmanındaki değişiklikler ve volume'daki kayıtlar, kaynak imaj registry'ye yüklendiğinde otomatik olarak gönderilmez [2], [12], [26].

Örneğin yerel veri tabanına eklenen öğrenci kayıtları bir volume'da tutuluyorsa, uygulama imajının buluta yüklenmesi bu kayıtları taşımaz. Hedef ortamda veri alanı ayrıca hazırlanmalı; mevcut veriler gerekli ise ayrı bir aktarım yapılmalıdır.

Ayrıca imajın işletim sistemi ve işlemci mimarisi hedef ortamla uyumlu olmalıdır [4]. Yereldeki Windows dosya yolları veya yalnız yerel erişim için kullanılan `localhost` ayarları da bulut ortamında aynı anlamı taşımayabilir. İmaj uygulama paketini taşır; hedef ortamın çalışma koşulları ayrıca düzenlenir.

8.2. Bulut Sanal Makinesinde Çalıştırma ve Yönetilen Hizmet Seçenekleri

Registry'de bulunan bir imajı bulutta çalıştırmanın iki temel yolu vardır: Bir sanal makinede container ortamını kendimiz yönetmek veya yönetilen bir container hizmeti kullanmak. Her iki yaklaşımda da uygulama imajı kullanılır; farklı olan, çalışma ortamının nasıl hazırlandığı ve kim tarafından yönetildiğidir.

Bulut sanal makinesinde Docker ve Compose:

İlk yaklaşımda Linux tabanlı bir bulut sanal makinesi hazırlanır ve üzerine Docker Engine ile Compose kurulur. Registry'den alınan imajlar, bu makinede container'lar oluşturmak için kullanılır. Yerelde öğrenilen servis, ağ ve volume ilişkileri aynı mantıkla uygulanır [41].

Örneğin web uygulaması ile veri servisi aynı sanal makinede Compose ile çalıştırılabilir. Web servisine erişim sağlanır; veri servisi uygulamanın ortak ağından kullanılır. Ancak yerel dosya yolları, erişim ayarları ve veri bağlantıları hedef ortama göre düzenlenmelidir [41].

Bu yaklaşımda, sanal makinenin işletim sistemi güncellemeleri, Docker ortamı, uygulama yapılandırması ve veri yönetimi takip edilir. Container'ın durması ile sanal makinenin kapanması ayrı olaylardır. Ayrıca bir makinede volume kullanmak, verilerin başka bir makineye otomatik taşınmasını sağlamaz.

Bu dersin bulut uygulamasında temel yol, yereldeki Docker/Compose yaklaşımının bir Linux sanal makinesinde uygulanmasıdır. Böylece öğrenci yeni bir dağıtım modelini öğrenmeden, bildiği kavramların buluttaki karşılığını görebilir.

Yönetilen container hizmetleri:

İkinci yaklaşımda sağlayıcı, container'ların çalışacağı altyapının belirli bölümlerini yönetir. Kullanıcı imajı ve gerekli çalışma ayarlarını tanımlar; uygulama hizmet üzerinden başlatılır. Yönetim kapsamı seçilen hizmete ve çalışma seçeneğine göre değişir.

Örneğin Amazon Web Services (AWS) üzerinde Amazon Elastic Container Service (ECS, Esnek Container Hizmeti), AWS Fargate ile kullanıldığında kullanıcı alttaki sunucuları yönetmeden container çalıştırabilir. Microsoft Azure'daki Azure Container Apps ve Google Cloud'daki Cloud Run da sunucu yönetimini kullanıcıdan soyutlayan seçenekler sunar [42]–[44]. Bu hizmetlerin özellikleri ve uygulama gereksinimleri birebir aynı değildir.

Sunucu yönetiminin azalması, uygulamanın bütün sorumluluklarının sağlayıcıya geçtiği anlamına gelmez. İmajın seçimi, uygulama ayarları, erişim izinleri ve kalıcı verinin nasıl yönetileceği yine belirlenmelidir.

Tablo 13, iki yaklaşımın temel farklarını özetlemektedir.

Tablo 13. Sanal makinede çalışma ile yönetilen container hizmetlerinin karşılaştırılması.

Boyut	Sanal makinede Docker/Compose	Yönetilen container hizmeti
Çalışma ortamı	Kullanıcı işletim sistemi ve container ortamını hazırlar.	Sağlayıcı altyapının belirli bölümlerini yönetir.
Uygulama tanımı	Docker ve Compose yapılandırması kullanılır.	Hizmetin desteklediği dağıtım tanımı kullanılır.
Kontrol düzeyi	İşletim sistemi ve Docker üzerinde doğrudan kontrol vardır.	Kontrol, hizmetin sunduğu ayarlarla sınırlıdır.
Kalıcı veri	Volume ve bağlı diskler kullanıcı tarafından düzenlenir.	Hizmetin desteklediği depolama veya harici veri hizmetleri seçilir.
Ders açısından rolü	Yerelde öğrenilen yaklaşımın buluta taşınması	Alternatif çalışma modelinin anlaşılması

Aynı imaj, farklı dağıtım tanımı:

Sanal makinede kullanılan `compose.yaml` dosyasının, yönetilen bir hizmette değişiklik yapılmadan çalışacağı varsayılmamalıdır. İmaj kullanılabilir olsa da servis bağlantıları, portlar, veri alanları ve kaynak ayarları hizmetin modeline göre yeniden tanımlanabilir [42]–[44].

Örneğin web uygulaması yönetilen bir hizmette çalıştırılırken veri tabanı ayrı bir veri hizmetinde tutulabilir. Bu durumda uygulamanın bağlantı adresi değişir. İmajın taşınabilir olması, dağıtım yapılandırmasının her ortamda aynı olması demek değildir. Sağlayıcıların kavramsal karşılıkları Bölüm 8.3'te ele alınmaktadır.

8.3. AWS, Microsoft Azure ve Google Cloud'da Kavramların Karşılıkları

Bu derste öğrenilen imaj saklama, container çalıştırma ve çalışma ortamını yönetme kavramları, bulut sağlayıcılarında farklı hizmet adlarıyla karşılık bulur. Tablo 14, bu kavramların üç sağlayıcıdaki örneklerini gösterir. Hizmetler benzer ihtiyaçları karşılamakla birlikte özellikleri ve yönetim modelleri tamamen aynı değildir.

Tablo 14. Derste öğrenilen kavramların üç bulut sağlayıcısındaki hizmet karşılıkları.

Kavram veya ihtiyaç	AWS	Microsoft Azure	Google Cloud
İmajları bir registry'de saklamak ve dağıtmak	Amazon Elastic Container Registry (ECR)	Azure Container Registry (ACR)	Artifact Registry
Bir bulut sanal makinesinde Docker Engine ile container çalıştırmak	Amazon Elastic Compute Cloud (EC2)	Azure Virtual Machines	Compute Engine
Sunucu yönetimini sağlayıcıya bırakarak container çalıştırmak	Amazon ECS ile AWS Fargate	Azure Container Apps	Cloud Run

AWS karşılıkları [45], Azure'un imaj saklama ve sanal makine hizmetleri [46], [47], Google Cloud'un ilgili hizmetleri [48], [49] numaralı kaynaklarda açıklanmıştır. Yönetilen container hizmetlerinin temel rolleri ise önceki bölümde kullanılan [42]–[44] numaralı kaynaklara dayanır.

Tabloyu yorumlarken imajın saklandığı hizmet ile container'ın çalıştığı hizmeti ayırmak gerekir. Örneğin AWS'de ECR imajı saklar; ECS ve Fargate bu imajdan container çalıştırmak için birlikte kullanılabilir. Azure'da ACR ile Azure Container Apps, Google Cloud'da ise Artifact Registry ile Cloud Run arasında benzer bir görev ayrımı vardır.

Sanal makine seçeneğinde öğrenci, yerelde öğrendiği Docker Engine ve Compose yaklaşımını bir bulut sunucusunda uygular. Yönetilen hizmet seçeneğinde ise çalıştırılacak imaj ve uygulamanın gereksinimleri, ilgili hizmetin yapılandırması üzerinden belirtilir. Her iki durumda da imaj, container, port, ortam değişkeni ve kalıcı veri kavramları geçerliliğini korur; bunların tanımlanma ve yönetilme biçimi çalışma ortamına göre değişir.

Bu bölüm için beklenen kazanım, hizmet adlarını ayrıntılarıyla ezberlemek değil, bir hizmetle karşılaşıldığında "İmajı mı saklıyor, sanal makine mi sağlıyor, yoksa container'ı mı çalıştırıyor?" sorusunu yanıtlayabilmektir.

8.4. Bulutta Erişim, Yapılandırma, Kalıcı Veri ve Kaynakların Sonlandırılması

Yerelde doğrulanan bir imajın bulutta çalıştırılması, uygulamanın erişim ve veri gereksinimlerinin yeni ortama göre düzenlenmesini gerektirir. Aynı imaj kullanılabilir;

ancak ağ adresleri, erişim izinleri, yapılandırma değerleri ve veri saklama alanları ortamdan ortama değişebilir.

Uygulamaya erişim:

Yerelde tarayıcıdan kullanılan `localhost`, öğrencinin kendi bilgisayarını gösterir. Uygulama buluta taşındığında öğrencinin bilgisayarındaki `localhost` bulut sunucusunu göstermez. Erişim için bulut sunucusunun uygun ağ adresi veya yönetilen hizmetin sağladığı uygulama adresi kullanılır.

Sanal makinede Docker ile çalışırken container portunun yayımlanması, erişim yolunun yalnızca bir parçasıdır. Sunucunun ağ üzerinden erişilebilir olması, bulut ağ kurallarının bağlantıya izin vermesi ve uygulamanın doğru adreste ve portta dinlemesi de gerekir. Yalnızca sunucunun `127.0.0.1` adresine yayımlanan bir port, doğrudan uzaktaki öğrencinin tarayıcısından erişilebilir hâle gelmez. [29]

Yönetilen hizmetlerde dış erişim, hizmetin sunduğu erişim ayarları üzerinden tanımlanır. Her iki yaklaşımda da web uygulamasına erişim sağlanırken veri servisinin doğrudan internete açılması gerekmez.

Çalışma ortamına uygun yapılandırma:

Veri servisi adresi, bağlantı portu ve uygulama ayarları çalışma anında sağlanmalıdır. Örneğin Compose ağında kullanılan `db` adı, o ağ içindeki veri servisini belirtir; farklı bir bulut ortamında aynı adın kendiliğinden geçerli olması beklenmez.

Bu nedenle ortam değişkenleri ve bağlantı bilgileri hedef ortama göre düzenlenir. Parolalar ve erişim anahtarları imajın içine yerleştirilmez. Uygulama paketini taşıyan imaj ile ortama özgü yapılandırmanın ayrılması, aynı imajın farklı ortamlarda kullanılmasını kolaylaştırır. [41]

Kalıcı verinin korunması:

Yerelde kullanılan bir volume, imaj registry'ye gönderildiğinde buluta taşınmaz. Bulutta veri saklama alanı ayrıca oluşturulur; mevcut verinin aktarılması gerekiyorsa bu işlem de ayrıca gerçekleştirilir. [26], [39]

Sanal makinedeki Docker volume, container silinse bile korunabilir; ancak verinin bulunduğu disk silinirse bu koruma yeterli olmaz. Yönetilen hizmetlerde de container'ın yazılabilir dosya sisteminin kalıcı olduğu varsayılmamalıdır. Örneğin Cloud Run'da bu alana yazılan veriler, container örneği sonlandırıldığında korunmaz. [50]

Dolayısıyla kalıcılık değerlendirilirken yalnızca "Container yeniden oluşturulursa veri kalır mı?" değil, "Sunucu veya çalışma ortamı kaldırılırsa veri nerede kalır?" sorusu da yanıtlanmalıdır.

Kullanılmayan kaynakların sonlandırılması:

Container'ı durdurmak, bulut sanal makinesini durdurmak ve bulut kaynaklarını silmek farklı işlemlerdir. Sanal makine üzerinde container'ların durdurulması, sanal makinenin çalışmasını veya ücretlendirilmesini kendiliğinden sonlandırmaz.

Sanal makine durdurulduğunda da disk gibi kaynakların ücretleri devam edebilir. Ayrıca Azure'da Stopped (durdurulmuş) durumundaki bir sanal makine ile Deallocated (kaynakları serbest bırakılmış) durumundaki bir sanal makinenin hesaplama ücretlendirmesi farklıdır. [51], [52]

Çalışma tamamlandığında uygulamanın yanında oluşturulan sanal makineler, diskler, registry'deki imajlar ve diğer yardımcı kaynaklar da gözden geçirilir. Gerekli veriler korunup yedeklendikten sonra kullanılmayan kaynaklar kaldırılır. Uygulamanın sona ermesi ile tüm bulut kaynaklarının sona ermesi aynı şey değildir.

9. Container Güvenliğinin Temelleri

Container'lar uygulamaların birbirinden ayrılmasını sağlar; ancak bu ayrım tek başına güvenli bir çalışma ortamını garanti etmez. Güvenlik, kullanılan imajın kaynağına ve güncelliğine, uygulamanın yetkilerine, hassas bilgilerin korunmasına ve dışarıya açılan erişimlere de bağlıdır. Bu bölümde, yerelde ve bulutta container çalıştırırken uygulanması gereken temel güvenlik ilkeleri ele alınır.

9.1. İmaj Güvenliği ve Kullanıcı Yetkileri

Container güvenliği, uygulama çalıştırılmadan önce imaj seçimiyle başlar ve çalışma sırasında verilen yetkilerle devam eder. Temel yaklaşım, kaynağı bilinen ve güncel bir imaj kullanmak, uygulamaya yalnızca ihtiyaç duyduğu yetkileri vermektir.

Güvenilir ve sade imajlar:

Bir imaj seçilirken yayıncısı, bakım durumu, belgeleri ve desteklenen sürümleri incelenmelidir. Docker Hub'daki Docker Official Images (Docker resmî imajları) veya yazılımın doğrulanmış yayıncısına ait imajlar uygun başlangıç noktalarıdır. Bununla birlikte güvenilir bir yayıncı, imajda hiçbir güvenlik açığı bulunmadığı anlamına gelmez.

İmaja yalnızca uygulamanın çalışması için gereken paketler eklenmelidir. Kullanılmayan araçlar ve kütüphaneler, bakım yükünü ve olası güvenlik açığı sayısını artırabilir. Amaç yalnızca küçük bir imaj üretmek değil, gereksiz bileşenleri azaltmaktır. [24]

Güncellemelerin uygulanması:

Temel imaj veya uygulama bağımlılıkları için yayımlanan güvenlik güncellemeleri, mevcut uygulama imajına kendiliğinden eklenmez. Güncellenmiş bileşenlerle imaj yeniden oluşturulur, uygulama doğrulanır ve çalışan container yeni imajdan oluşturulan container ile değiştirilir. Eski container'ı yeniden başlatmak bu güncellemeyi sağlamaz.

Önceki bölümlerde açıklanan digest ile sabitleme, kullanılan imaj içeriğini kesin olarak belirler; ancak güncelliği garanti etmez. Sabitlenen imajın güncellemeleri de takip edilmelidir. Tekrarlanabilirlik ile güvenlik güncellemelerinin uygulanması birlikte yönetilmelidir. [24]

Uygulamayı sınırlı yetkilerle çalıştırmak:

Linux tabanlı container'larda root (süper kullanıcı) geniş yetkilere sahiptir. Uygulamanın bu yetkilere ihtiyacı yoksa non-root (root olmayan kullanıcı) ile çalıştırılması tercih edilir. Bu yaklaşım, least privilege (en az yetki) ilkesinin uygulanmasıdır: uygulamaya görevini tamamlamak için gereken izinler verilir.

Dockerfile'daki `USER` yönergesi, sonraki ilgili işlemler ve container içinde çalıştırılan uygulama için kullanılacak kullanıcıyı belirler. Paket kurulumu sırasında gereken yetkiler ile uygulamanın çalışma sırasında ihtiyaç duyduğu yetkiler aynı olmak zorunda değildir. [21]

Örneğin web uygulamasının kendi dosyalarını okuyabilmesi ve belirlenen veri dizinine yazabilmesi yeterli olabilir. Sistemdeki her dosyayı değiştirebilmesine gerek yoktur. Volume veya bind mount kullanıldığında da dosya izinleri bu kullanıcıyla uyumlu olmalıdır. Bir izin hatasını çözmek için uygulamayı doğrudan root olarak çalıştırmak yerine, gerekli dizinin sahipliği ve erişim izinleri değerlendirilir.

Container içindeki root kullanıcısı, olağan yalıtım koşullarında ana makinede sınırsız erişim anlamına gelmez. Bununla birlikte container'lar ana makinenin çekirdeğini paylaşır; geniş yetkiler ve ana makinenin hassas alanlarına verilen erişimler yalıtımın koruyuculuğunu azaltabilir. Root olmayan kullanıcıyla çalışmak da tek başına tam güvenlik sağlamaz. [3]

9.2. Hassas Bilgiler ve Güvenlik Kontrolleri

Güncel ve güvenilir bir imaj kullanmak, güvenliğin yalnızca bir parçasıdır. Parolaların nasıl sağlandığı, container'ın hangi kaynaklara erişebildiği ve güvenlik bulgularının nasıl değerlendirildiği de önemlidir.

Hassas bilgileri imajdan ayrı tutmak:

Secret (gizli bilgi); parola, erişim anahtarı veya özel sertifika gibi korunması gereken değerleri ifade eder. Bu bilgiler kaynak koduna, Dockerfile'a veya imaja kopyalanan dosyalara yazılmamalıdır. İmaja erişebilen bir kişi, içindeki hassas bilgilere de erişebilir. Derleme sırasında gereken gizli bilgiler için de bunları imaja kaydetmeyen özel mekanizmalar kullanılmalıdır. [53], [54]

Ortam değişkenleri, yapılandırmayı imajdan ayırmayı sağlar; ancak bir değeri ortam değişkeni olarak vermek, onu kendiliğinden gizli veya şifreli yapmaz. Değerler hata

ayıklama çıktılarında, günlüklerde veya yetkili kullanıcıların incelemelerinde açığa çıkabilir. Benzer şekilde .env dosyası da tek başına bir güvenlik mekanizması değildir.

Bu derste kullanılan Linux container'larında Compose'un secrets özelliği, gizli bilgileri yalnızca izin verilen servislere dosya olarak sunabilir. Uygulamanın da ilgili değeri bu dosyadan okuyabilmesi gerekir. Yerelde kaynak olarak kullanılan gizli bilgi dosyasının erişim izinleri ayrıca korunmalı ve dosya sürüm kontrolüne eklenmemelidir; Compose kullanılması bu dosyayı kendiliğinden şifrelemez. [53]

Çalışma sırasında erişimi sınırlandırmak:

Önceki bölümdeki en az yetki ilkesi, kullanıcı hesabının yanında container'ın erişebildiği kaynaklara da uygulanır:

- Dosya erişimi: Yalnızca gerekli dizinler bağlanır. Yazılması gerekmeyen alanlar salt okunur sunulabilir.
- Ağ erişimi: Yalnızca gerekli portlar yayımlanır. Web uygulamasının kullandığı veri servisi, ihtiyaç yoksa dış erişime açılmaz.
- Sistem yetkileri: Container'a gereksiz ek yetkiler veya privileged mode (ayrıcalıklı çalışma modu) verilmez. Docker Engine'i kontrol eden bağlantının container'a açılması da sıradan bir dosya paylaşımı olarak değerlendirilmez; geniş yönetim yetkileri sağlayabilir. [3], [27], [29]

Örneğin web uygulamasının veri servisine bağlanabilmesi için ana makinenin bütün dosyalarına veya Docker Engine'in yönetimine erişmesi gerekmez. Bir bağlantı ya da izin hatasında tüm kısıtlamaları kaldırmak yerine, eksik olan belirli izin bulunmalıdır.

Güvenlik taramalarını yorumlamak:

Vulnerability scanning (güvenlik açığı taraması), imajdaki yazılım bileşenlerini bilinen güvenlik açıklarıyla karşılaştırır. Örneğin Docker Scout, imajın bileşen envanterini çıkarıp güvenlik açığı veritabanlarıyla eşleştirir. [55]

Bir bulgu değerlendirilirken etkilenen paket ve sürüm, önem düzeyi, düzeltilmiş sürümün bulunup bulunmadığı ve bileşenin uygulamada nasıl kullanıldığı incelenir. Uygun düzeltme sonrasında imaj yeniden oluşturulur, uygulama doğrulanır ve tarama tekrarlanır.

Tarama sonucunda açık bulunmaması, uygulamanın tamamen güvenli olduğu anlamına gelmez. İmaj taraması; yanlış erişim izinlerinin, dışarıya açılmış veri servisinin veya uygulama mantığındaki bütün hataların yerini tutmaz. Bu nedenle tarama sonuçları, yapılandırma ve yetki kontrolleriyle birlikte değerlendirilir.

10. Uygulama Çalışması

Bu çalışmada, hazır bir web uygulaması ve veri servisi kullanılarak haftanın kavramları uygulamaya dönüştürülür. Yerel çalışma tüm öğrenciler için zorunludur. Bulut erişimi olan öğrencilerden aynı uygulamayı bir bulut sağlayıcısında da çalıştırmaları beklenir. Kodlar ve adım adım yönergeler *Ek 2 — Yerel Container Uygulama Kılavuzu* ve *Ek 3 — Container Uygulamasını Bulutta Çalıştırma Kılavuzu* içinde verilmiştir.

10.1. Uygulamanın Amacı ve Kapsamı

Çalışmanın amacı, öğrencinin hazır bir uygulamayı imaja dönüştürmesi, ağ ve veri gereksinimleriyle çalıştırması ve aynı imajın bulutta kullanımını anlamasıdır.

Uygulama, tarayıcı üzerinden kayıt ekleme ve listeleme işlemlerini gerçekleştiren bir web bileşeni ile kayıtları saklayan ayrı bir veri servisinden oluşur. Öğrenci uygulamanın kaynak kodlarını hazır kullanır; imaj oluşturma ve çalışma ortamını düzenleme görevlerini üstlenir.

Çalışma sonunda öğrencinin:

- Uygulama imajını oluşturması ve web uygulaması ile veri servisini Compose kullanarak birlikte çalıştırması,
- Tarayıcıdan erişimi ve servisler arası bağlantıyı doğrulaması; container yeniden oluşturulduğunda kayıtların korunduğunu göstermesi,
- İmaj, yapılandırma ve kalıcı veriyi birbirinden ayırarak buluta taşıma gereksinimlerini açıklaması

beklenir. Bulut çalışmasını gerçekleştiren öğrenciler ayrıca yerelde doğruladıkları uygulama imajını seçtikleri sağlayıcıda çalıştırır ve erişimini doğrular.

Uygulama boyunca gerekli kullanıcı yetkileri, hassas bilgilerin korunması ve kullanılmayan kaynakların kaldırılması da gözetilir.

10.2. Yerel Uygulama

Yerel uygulama tüm öğrenciler için zorunludur. Çalışma, Ek 1'de hazırlanan Windows ortamında, Docker Desktop kullanılarak Linux container'ları üzerinde gerçekleştirilir. Dosyaların hazırlanması, komutlar ve sorun giderme adımları *Ek 2 — Yerel Container Uygulama Kılavuzu*'nda verilmiştir.

Öğrenci, hazır web uygulaması ve veri servisi için aşağıdaki görevleri gerçekleştirir:

1. **Uygulama imajını oluşturma:** Dockerfile ve .dockerignore dosyalarını hazırlar; uygulamanın bağımlılıklarını ve başlatılma biçimini tanımlayarak sürüm etiketi verilmiş bir imaj oluşturur.

2. **Bileşenleri birlikte çalıştırma:** Compose dosyasında web uygulamasını ve veri servisini tanımlar. Tarayıcıdan erişim için web portunu yayımlar; servisler arası bağlantı ayarlarını ve veri servisi için named volume kullanımını düzenler.
3. **İşlevleri ve veri kalıcılığını doğrulama:** Tarayıcıdan kayıt ekler ve listeler. Container'ları kaldırıp aynı volume'u koruyarak yeniden oluşturur; önceki kayıtların erişilebilir olduğunu kontrol eder.
4. **Uygulamayı güncelleme:** Ek 2'de belirtilen küçük uygulama değişikliğini gerçekleştirir, yeni sürüm etiketiyle imajı yeniden oluşturur ve web container'ını yeni imajla değiştirir. Güncellemenin uygulamaya yansıdığını ve kayıtların korunduğunu doğrular.

Çalışma sırasında web uygulaması, veri servisine Compose ağındaki servis adı üzerinden bağlanır. Veri servisinin portunun ana makineye yayımlanması gerekmez. Hassas bilgiler imajın içine yerleştirilmez.

Başarı ölçütü yalnızca container'ların çalışıyor görünmesi değildir. Uygulamanın tarayıcıdan kullanılabilmesi, veri servisiyle iletişim kurması ve container'lar yeniden oluşturulduğunda kayıtların korunması birlikte gösterilmelidir.

Doğrulama tamamlandıktan sonra çalışan kaynaklar kontrollü biçimde sonlandırılır. Sonraki haftada kullanılacak uygulama dosyaları, Dockerfile ve Compose dosyası korunur. Volume'un korunması ile silinmesinin sonuçları ayırt edilerek veri kaldırma işlemi yalnızca bilinçli olarak gerçekleştirilir.

10.3. Bulut Uygulaması

Bulut erişimi olan öğrencilerden, yerelde doğruladıkları uygulamayı AWS, Microsoft Azure veya Google Cloud sağlayıcılarından birinde çalıştırmaları beklenir. Bir sağlayıcıda çalışma yapılması yeterlidir. Bulut hesabı veya gerekli kaynaklara erişimi bulunmayan öğrenciler için yerel uygulamanın tamamlanması zorunlu kapsamı karşılar.

Bu çalışmada amaç, yerelde oluşturulan uygulama imajını bulutta kullanmak ve çalışma ortamına bağlı gereksinimleri ayırt etmektir. Sağlayıcıya özgü işlemler *Ek 3 — Container Uygulamasını Bulutta Çalıştırma Kılavuzu*'nda verilmiştir.

Öğrenci aşağıdaki görevleri gerçekleştirir:

1. **İmajı erişilebilir hâle getirme:** Yerelde doğrulanan uygulama imajını bir registry'ye gönderir. Bulut ortamının bu imajı çekebilmesi için gerekli erişim izinlerini düzenler ve kullanılan imaj sürümünü kaydeder.
2. **Uygulamayı bulutta çalıştırma:** Ek 3'te seçilen sağlayıcıya ait yolu izler. Temel uygulama yolu, bir Linux sanal makinesinde Docker Engine ve Compose ile web uygulamasını ve veri servisini birlikte çalıştırmaktır.

3. **Erişim ve veri gereksinimlerini düzenleme:** Web uygulamasına uzaktan erişimi sağlar; servisler arası bağlantıyı, ortama özgü yapılandırmayı ve kalıcı veri alanını tanımlar. Veri servisini ihtiyaç olmadan internete açmaz.
4. **Doğrulama ve kaynakları kaldırma:** Tarayıcıdan kayıt ekleme ve listeleme işlemlerini doğrular. Veri alanını koruyarak container'ları yeniden oluşturur ve kayıtların korunduğunu gösterir. Kanıtları aldıktan sonra gerekli verileri korur ve kullanılmayan bulut kaynaklarını kaldırır.

Bulut ortamında yeni test kayıtları oluşturulması yeterlidir; yereldeki kayıtların buluta aktarılması bu çalışmanın zorunlu kapsamına girmez. Ancak öğrenci, imajın registry'ye gönderilmesinin volume içindeki verileri taşımadığını açıklayabilmelidir.

Yönetilen container hizmetleri, Ek 3'te ek bir çalışma yolu olarak değerlendirilebilir. Bu yol seçildiğinde ağ, yapılandırma ve kalıcı veri gereksinimleri ilgili hizmete uygun biçimde karşılanır.

Başarı ölçütü, aynı uygulama imajının bulutta çalıştığının, uygulamaya erişilebildiğinin ve veri işlemlerinin gerçekleştiğinin gösterilmesidir. Container'ların durdurulmasıyla tüm bulut kaynaklarının kaldırılmasının farklı işlemler olduğu da dikkate alınır.

10.4. Doğrulama ve Teslim

Öğrenci, uygulamanın işlevlerini ve veri kalıcılığını gözlemlenebilir kanıtlarla doğrular. İmajın oluşturulması veya container'ların çalışıyor görünmesi tek başına yeterli değildir.

Tablo 15. Uygulama çalışmasında doğrulanması gereken sonuçlar.

Kontrol	Beklenen sonuç
İmaj ve çalışma durumu	Uygulama imajının sürüm etiketi ve çalışan web/veri servisi container'ları gösterilir.
Uygulamaya erişim	Tarayıcıdan web arayüzüne erişilir; kayıt ekleme ve listeleme işlemleri gerçekleştirilir.
Veri kalıcılığı	Aynı volume korunarak container'lar kaldırılıp yeniden oluşturulduğunda önceki kayıtlar görüntülenir.
Uygulama güncellemesi	Yeni imajdan oluşturulan web container'ında yapılan değişiklik görülür; mevcut kayıtlar korunur.
Bulut çalışması (gerçekleştiren öğrenciler için)	Yerelde doğrulanan uygulama imajı bulutta çalıştırılır; uzaktan erişim ve veri işlemleri gösterilir.
Kaynakların sonlandırılması	Kullanılmayan çalışma kaynaklarının kaldırıldığı, hangi veri alanlarının korunduğu veya silindiği belirtilir.

Veri kalıcılığı kanıtında, yeniden oluşturma öncesinde ve sonrasında aynı kayıtların bulunduğu açıkça görülmelidir. Container'ı yalnızca durdurup başlatmak bu kontrolün yerine geçmez.

Teslim içeriği

Öğrenci, aşağıdaki iki bileşenden oluşan kısa bir uygulama raporu teslim eder:

- **Doğrulama kanıtları:** Tablo 15'teki sonuçları gösteren ekran görüntüleri veya ilgili komut çıktıları. Her kanıtın altında, hangi sonucu gösterdiğini belirten kısa bir açıklama bulunur. Bulut çalışması yapan öğrenciler seçtikleri sağlayıcıyı, kullandıkları imaj sürümünü ve kaynakların kaldırılmasına ilişkin kanıtları da ekler.
- **Kavramsal açıklamalar:** Öğrenci aşağıdaki soruları kendi uygulamasındaki değerleri ve gözlemleri kullanarak yanıtlar.

1. Tarayıcının bağlandığı ana makine portu ile web container'ının dinlediği port hangileridir? Web uygulaması veri servisine hangi adresle bağlanır?
2. Kayıtlar nerede saklanır? Container'lar yeniden oluşturulduğunda neden korunur? İlgili volume silinirse ne olur?
3. Uygulama değişikliğinin görünmesi için hangi imaj ve container işlemleri yapılmıştır? Eski container'ı yeniden başlatmak neden yeterli değildir?

Dockerfile, .dockerignore, Compose dosyası ve hazır kaynak kodların ayrıca teslim edilmesi zorunlu değildir. Sorunların incelenmesi gerektiğinde ilgili dosyalar istenebilir. Öğrenci, çalışma dosyalarını sonraki haftanın uygulamasında kullanmak üzere kendi bilgisayarında korur.

Parolalar, erişim anahtarları ve diğer hassas bilgiler raporda veya ekran görüntülerinde bulunmamalıdır.

Değerlendirmede, uygulamanın doğru çalışması ile öğrencinin imaj, container, ağ, yapılandırma ve kalıcı veri arasındaki ilişkiyi kendi çalışması üzerinden açıklayabilmesi birlikte dikkate alınır.

10.5. Sonraki Haftaya Hazırlık

Bu haftaki uygulama, sonraki haftanın Kubernetes çalışmasının temelini oluşturur. Aynı web uygulaması ve veri servisi kullanılarak, container'ların farklı bir yönetim ortamında çalıştırılması ele alınır.

Öğrenci aşağıdaki materyalleri kendi bilgisayarında korur:

- Hazır uygulama kaynakları ve güncellenmiş uygulama dosyaları,
- Dockerfile, .dockerignore ve Compose dosyası,
- Kullanılan imaj sürümü, portlar, bağlantı ayarları ve veri saklama gereksinimlerine ilişkin notlar.

Çalışan container'ların veya ücretli bulut kaynaklarının açık tutulması gerekmez. Korunan dosyalar, uygulama imajının yeniden oluşturulmasına ve çalışma ortamının yeniden kurulmasına olanak sağlar. Hassas bilgiler paylaşılabilir dosyalardan ayrı tutulur.

Sonraki çalışmaya hazırlık için öğrencinin web ve veri servisinin görevlerini, birbirlerine nasıl bağlandıklarını ve hangi verinin kalıcı olması gerektiğini açıklayabilmesi yeterlidir. Bu haftanın uygulamasında Kubernetes kurulumu veya Kubernetes yapılandırma dosyalarının hazırlanması beklenmez.

11. Bölüm Özeti

Bu hafta, hazır bir uygulamanın container imajına dönüştürülmesi, ağ ve veri gereksinimleriyle çalıştırılması ve aynı imajın bulutta kullanılması ele alınmıştır.

- İmaj ve container farklıdır. İmaj, uygulamayı ve bağımlılıklarını içeren katmanlı pakettir. Container, bu imajdan oluşturulan çalışma örneğidir ve kendine ait yazılabilir katmana sahiptir. Aynı imajdan birden fazla container oluşturulabilir.
- Dockerfile, imajın nasıl oluşturulacağını tanımlar. Derleme bağlamı kullanılabilir dosyaları belirler; `.dockerignore` gereksiz dosyaları bu bağlamın dışında tutar. Uygulama değişikliklerinin çalışmaya yansıması için yeni imaj oluşturulması ve container'ın bu imajdan yeniden oluşturulması gerekir.
- Ağ erişimi, bağlantının hangi bileşenler arasında kurulduğuna göre düzenlenir. Tarayıcıdan erişim için port yayımlama kullanılırken aynı Compose ağındaki servisler birbirlerine servis adları ve container portları üzerinden bağlanabilir. `localhost`, bağlantıyı başlatan ortamı ifade eder.
- Kalıcı veri, container'ın yaşam döngüsünden ayrılmalıdır. Container'ın yazılabilir katmanı container kaldırıldığında silinir. Volume kullanılması, verinin container yeniden oluşturulduğunda korunmasını sağlar; ancak yedeklemenin yerini tutmaz.
- Compose, çok bileşenli uygulamanın birlikte yönetilmesini sağlar. Servisler, ağlar, veri alanları ve yapılandırma tek bir dosyada tanımlanır. Bir container'ın çalışıyor olması, uygulamanın bağlantı kabul etmeye hazır veya işlevsel olduğunu tek başına göstermez.
- Buluta taşınabilirlik, yalnızca imajın aktarılması değildir. Registry imajı saklar ve dağıtır; çalışma ortamı container'ı çalıştırır. Erişim izinleri, yapılandırma ve kalıcı veri ayrıca düzenlenir. Yerel volume içeriği, imajla birlikte buluta taşınmaz.
- Güvenlik ve kaynak yönetimi çalışma sürecinin parçasıdır. Güvenilir ve güncel imajlar kullanılır, yetkiler sınırlandırılır, hassas bilgiler imajdan ayrı tutulur ve güvenlik bulguları değerlendirilir. Kullanılmayan container'lar ve bulut kaynakları kontrollü biçimde kaldırılır.

Haftanın temel kazanımı, uygulama imajı, çalışma yapılandırması ve kalıcı veriyi birbirinden ayırarak uygulamayı yeniden kurulabilir ve doğrulanabilir biçimde çalıştırabilmektir.

Kaynakça

[1] “What is a container?,” Docker Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/
[2] “What is an image?,” Docker Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/
[3] “Docker Engine security,” Docker Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.docker.com/engine/security/
[4] “Multi-platform builds,” Docker Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.docker.com/build/building/multi-platform/
[5] “Docker Engine,” Docker Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.docker.com/engine/
[6] “Docker Desktop,” Docker Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.docker.com/desktop/
[7] “Docker Compose,” Docker Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.docker.com/compose/
[8] “Install Docker Desktop on Windows,” Docker Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.docker.com/desktop/setup/install/windows-install/
[9] “Docker Desktop WSL 2 backend on Windows,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/desktop/features/wsl/
[10] “Install Docker Engine on Ubuntu,” Docker Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.docker.com/engine/install/ubuntu/
[11] “Install Docker Desktop on Mac,” Docker Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.docker.com/desktop/setup/install/mac-install/
[12] “Storage drivers,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/engine/storage/drivers/
[13] “What is a registry? | Docker Docs.” Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-registry/
[14] “docker image tag,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/image/tag/
[15] “Image digests,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/dhi/explore/security-concepts/digests/
[16] “Running containers,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/engine/containers/run/
[17] “docker container create,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/container/create/
[18] “docker container stop,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/container/stop/

[19] “docker container rm,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/container/rm/
[20] “Dockerfile overview,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/build/concepts/dockerfile/
[21] “Dockerfile reference,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/reference/dockerfile/
[22] “Build context,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/build/concepts/context/
[23] “Docker build cache,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/build/cache/
[24] “Building best practices,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/build/building/best-practices/
[25] “Storage,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/engine/storage/
[26] “Volumes,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/engine/storage/volumes/
[27] “Bind mounts,” Docker Documentation. Accessed: Oct. 01, 2026. [Online]. Available: https://docs.docker.com/engine/storage/bind-mounts/
[28] “Networking overview,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/engine/network/
[29] “Port publishing and mapping,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/engine/network/port-publishing/
[30] “Bridge network driver,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/engine/network/drivers/bridge/
[31] “How Compose works,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/compose/intro/compose-application-model/
[32] “Define services in Docker Compose,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/reference/compose-file/services/
[33] “Networking in Compose,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/compose/how-tos/networking/
[34] “Control startup and shutdown order in Compose,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/compose/how-tos/startup-order/
[35] “docker compose up,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/compose/up/
[36] “docker compose restart,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/compose/restart/
[37] “docker compose stop,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/compose/stop/
[38] “docker compose down,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/compose/down/
[39] “docker image push,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/image/push/
[40] “docker image pull,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/image/pull/

[41] “Use Compose in production,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/compose/how-tos/production/
[42] “Architect your solution for Amazon ECS - Amazon Elastic Container Service.” Accessed: Oct. 02, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/ecs-configuration.html
[43] craigshoemaker, “Azure Container Apps overview.” Accessed: Oct. 02, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/container-apps/overview
[44] “What is Cloud Run,” Google Cloud Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.cloud.google.com/run/docs/overview/what-is-cloud-run
[45] “Choosing an AWS container service - AWS Decision Guides.” Accessed: Oct. 02, 2026. [Online]. Available: https://docs.aws.amazon.com/decision-guides/latest/decision-guides/choosing-aws-container-service.html
[46] KumudD, “Introduction to Azure Container Registry - Azure Container Registry.” Accessed: Oct. 02, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/container-registry/container-registry-intro
[47] mattmcinnes, “Overview of virtual machines in Azure - Azure Virtual Machines.” Accessed: Oct. 02, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/virtual-machines/overview
[48] “Artifact Registry overview,” Google Cloud Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.cloud.google.com/artifact-registry/docs/overview
[49] “Compute Engine overview,” Google Cloud Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.cloud.google.com/compute/docs/overview
[50] “Container runtime contract | Cloud Run,” Google Cloud Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.cloud.google.com/run/docs/container-contract
[51] “Stop and start Amazon EC2 instances - Amazon Elastic Compute Cloud.” Accessed: Oct. 02, 2026. [Online]. Available: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/Stop_Start.html
[52] mimckitt, “States and billing status - Azure Virtual Machines.” Accessed: Oct. 02, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/virtual-machines/states-billing
[53] “Manage secrets securely in Docker Compose,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/compose/how-tos/use-secrets/
[54] “Build secrets,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/build/building/secrets/
[55] “Docker Scout,” Docker Documentation. Accessed: Oct. 02, 2026. [Online]. Available: https://docs.docker.com/scout/

Hafta 5 – Çalışma Soruları

A. Container ve İmaj Kavramları

1. Container (konteyner) ile sanal makine arasındaki temel farkları açıklayınız. İşletim sistemi çekirdeğinin paylaşılması, kaynak kullanımı ve yalıtım açısından bu iki yaklaşımı karşılaştırınız.
2. Bir imajdan iki container oluşturulduğunu düşününüz. Container'lardan birinin içinde uygulamaya ait bir dosya değiştirilmiştir. Bu değişiklik imajı veya diğer container'ı etkiler mi? Yanıtınızı imajın salt okunur katmanları ve container'ın yazılabilir katmanı üzerinden açıklayınız.
3. Windows kullanan bir öğrenci, ders uygulamasını Linux container'larıyla çalıştıracaktır. Docker Desktop ve Windows Subsystem for Linux 2 — WSL 2 (Linux için Windows Alt Sistemi 2) bu çalışma ortamında hangi rolleri üstlenir? Öğrencinin bilgisayarında Windows bulunmasına rağmen Linux tabanlı bir imajın nasıl çalıştırılabildiğini açıklayınız.

B. Uygulama İmajının Oluşturulması ve Dağıtılması

4. Dockerfile içindeki `FROM`, `COPY`, `RUN` ve `CMD` yönergelerinin görevlerini açıklayınız. Uygulamanın bağımlılıklarının yüklenmesi ile uygulamanın başlatılması neden farklı aşamalarda gerçekleştirilir?
5. Build context (derleme bağlamı) nedir? Bir proje klasöründeki her dosyanın imaj oluşturma sürecine gönderilmesi neden uygun olmayabilir? `.dockerignore` dosyasında dışlanması gereken üç dosya veya klasör türünü, gerekçeleriyle belirtiniz.
6. Bir container registry (konteyner imaj deposu) içindeki imajın adı, tag (etiket) değeri ve digest (içerik özeti) hangi amaçlarla kullanılır? Aynı etiketin daha sonra farklı bir imajı göstermesi mümkün müdür? Belirli bir imaj sürümünü kesin olarak seçmek için hangi bilgi kullanılmalıdır?

C. Container Yaşam Döngüsü ve Ağ Erişimi

7. Bir container'ı durdurmak, yeniden başlatmak ve silmek arasındaki farkları açıklayınız. Bu işlemlerin container'ın yazılabilir katmanındaki veriler üzerindeki etkilerini karşılaştırınız.
8. Bir öğrenci uygulamanın kaynak kodunu değiştirdikten sonra yalnızca mevcut container'ı yeniden başlatmıştır. Ancak uygulama hâlâ eski sürümü göstermektedir. Kaynak kodun imaja kopyalandığı bir yapıda bu durumun nedenini açıklayınız ve yeni sürümün çalıştırılması için gereken aşamaları sıralayınız.

9. Bir uygulama container içinde `8000` numaralı portta çalışmakta, bilgisayarda ise `127.0.0.1:8080` adresi üzerinden erişime açılmaktadır. Bu iki portun rollerini açıklayınız. Dockerfile içinde yalnızca `EXPOSE 8000` bulunması, uygulamaya bilgisayarın tarayıcısından erişmek için yeterli midir?

D. Container'lar Arası İletişim ve Veri Kalıcılığı

10. Aynı Docker ağına bağlı `web` ve `db` adlı iki servis bulunmaktadır. Web uygulamasının veritabanı adresi neden `localhost` yerine `db` olarak ayarlanmalıdır? Container içindeki `localhost` adresinin hangi ortamı gösterdiğini açıklayınız.

11. Bir veritabanı container'ının verileri yalnızca kendi yazılabilir katmanında tutulmaktadır. Container silinip yeniden oluşturulduğunda hangi sorun ortaya çıkabilir? Volume (veri birimi) kullanılması bu sorunu nasıl çözer?

12. Bind mount (ana makine dizini bağlama) ile volume arasındaki farkı açıklayınız. Kaynak kod üzerinde yerelde çalışmak ve veritabanı verilerini kalıcı tutmak için bu seçeneklerden hangilerini tercih edersiniz? Ayrıca, verilerin volume üzerinde tutulması neden tek başına yedekleme sayılmaz?

E. Çok Servisli Uygulamalar ve Güvenlik

13. Web uygulaması ve veritabanından oluşan bir sistemin Docker Compose ile tanımlanması hangi kolaylıkları sağlar? Compose dosyasındaki servis, ağ ve volume tanımlarının görevlerini açıklayınız.

14. Bir veritabanı container'ının çalışıyor olması, veritabanının bağlantı kabul etmeye hazır olduğu anlamına gelir mi? Health check (sağlık denetimi) ve `depends_on` kullanımı başlangıç sırasını nasıl destekler? Uygulama çalışırken veritabanı daha sonra erişilemez hâle gelirse, başlangıçta tanımlanan bağımlılık neden tek başına yeterli değildir?

15. Bir öğrenci veritabanı parolasını Dockerfile'a yazmış, uygulamayı `root` kullanıcısıyla çalıştırmış ve imajı bir depoya göndermiştir. Bu tercihler hangi güvenlik sorunlarını doğurabilir? Gizli bilgilerin yönetimi, kullanıcı yetkileri ve imaj güvenliği bakımından uygun iyileştirmeleri açıklayınız.

F. Bulutta Çalıştırma ve Bütünleşik Değerlendirme

16. Yerelde oluşturulan bir uygulama imajının buluttaki bir sanal makinede çalıştırılması için gereken temel aşamaları sıralayınız. İmajın depoya gönderilmesi, yereldeki veritabanı kayıtlarının da buluta taşındığı anlamına gelir mi? İmaj, çalışma yapılandırması ve kalıcı verilerin rollerini ayrı ayrı açıklayınız.

17. Container'ları buluttaki bir sanal makinede çalıştırmak ile yönetilen bir container hizmetinde çalıştırmak arasında kullanıcı sorumlulukları bakımından hangi farklar

vardır? İşletim sistemi bakımı, uygulama yapılandırması, veri kalıcılığı ve kaynakların kullanım sonunda temizlenmesi üzerinden karşılaştırınız.

18. Bir öğrenci kulübü için ekipman ödünç verme uygulaması hazırlanmıştır. Sistem, bir web servisi ve bir veritabanı servisinden oluşmaktadır. Uygulamanın önce Windows bilgisayarda, daha sonra bulutta çalıştırılması istenmektedir. Aşağıdaki gereksinimleri karşılayan bir çalışma düzenini açıklayınız:
    - Web servisi veritabanına erişebilmeli; veritabanı doğrudan dış erişime açılmamalıdır.
    - Container'lar silinip yeniden oluşturulduğunda kayıtlar korunmalıdır.
    - Parolalar uygulama imajına eklenmemelidir.
    - Yeni uygulama sürümü yayımlanırken veritabanı kayıtları korunmalıdır.
    - Buluttaki deneme tamamlandığında kullanılan kaynaklar temizlenmelidir.

Yanıtınızda gerekli servisleri, ağ erişimini, veri saklama yöntemini ve sürüm güncelleme yaklaşımını belirtiniz. Komut veya yapılandırma dosyası yazmanız gerekli değildir.
